

Python Programming - 2301CS404

Lab - 2

Jadeja Vishwrajsinh

24010101107

01) WAP to print "Hello World..!!"

```
In [2]: print("Hello World")
```

Hello World

02) WAP to accept your name and display a welcome message.

Input: Priya

Output: Hello Priya, welcome to Python Lab.

```
In [9]: name = input("Enter Your name :")  
print(f"Hello {name}, welcome to Python Lab")
```

Enter Your name : Priya

Hello Priya, welcome to Python Lab

03) WAP to accept three integers and display the numbers, their sum, and average.

Input: 10 20 30

Output:

Numbers: 10 20 30

Sum: 60

Average: 20.0

```
In [5]: a,b,c = input("Enter three numbers :").split()  
a = int(a)  
b = int(b)
```

```
c = int(c)
sum = a+b+c
avg = float(sum/3)
print("Sum :",sum)
print("Avg :",avg)
```

Enter three numbers : 10 20 30

Sum : 60

Avg : 20.0

04) WAP to accept name (string), age (int), and percentage (float).

Input : Riya,18,92.5

Output :

Name: Riya Type: <class 'str'>

Age: 18 Type: <class 'int'>

Percentage: 92.5 Type: <class 'float'>

```
In [8]: Name, Age, Percentage = input("Enter Name, age and percentage").split()
Age = int(Age)
Percentage = float(Percentage)
print(f"Name: {Name} Type: {type(Name)}")
print(f"Age: {Age} Type: {type(Age)}")
print(f"Percentage: {Percentage} Type: {type(Percentage)}")
```

Enter Name, age and percentage Riya 18 92.5

Name: Riya Type: <class 'str'>

Age: 18 Type: <class 'int'>

Percentage: 92.5 Type: <class 'float'>

05) WAP to print following message using custom separator and end.

Output : Python | Programming | Basics###

```
In [10]: print("Python", "Programming", "Basics###", sep=' | ')
```

Python | Programming | Basics###

06) WAP to accept a value and display its value, type, and memory id.

Input : hello

Output :

Value: hello

Type: <class 'str'>

ID: 140712345678912

```
In [12]: a = input("Enter Msg:")
print("Value:",a)
print("Type:", type(a))
print("ID:", id(a))
```

Enter Msg: hello
Value: hello
Type: <class 'str'>
ID: 2059143210176

07) WAP to assign a value to a variable, print id, reassign a new value, and print id again.

Output :

Original ID of a: 140712345678912

New ID of a: 140712345678960

```
In [13]: a = 10
print("Original ID of a:",id(a))
a = 20
print("New ID of a:",id(a))
```

Original ID of a: 140735217382600
New ID of a: 140735217382920

08) WAP to print multiple lines using a single print().

Output:

Welcome to Python
This is the second lab
Enjoy coding!

```
In [14]: print("Wwlcome to python\nThis is the second lab\nEnjoy coding!")
```

Wwlcome to python
This is the second lab
Enjoy coding!

09) WAP to display following table of items with proper alignment.

Output :

Sr No	Name	Subject	Grade	Percentage
1	Nisha Patel	Math	A	92
2	Aarav Modi	Science	B+	85
3	Jiya Shah	English	A+	96

```
In [21]: print(f"{'Sr No':<6} {'Name':^12} {'Subject':<10} {'Grade':<6} {'Percentage':<10}")
print(f"{1:>6} {'Nisha Patel':<12} {'Math':<10} {'A':^6} {92:>10}")
```

```
print(f"{2:>6} {'Aarav Modi':<12} {'Science':<10} {'B+':^6} {85:>10}")
print(f"{3:>6} {'Jiya Shah':<12} {'English':<10} {'A+':^6} {96:>10}")
```

Sr No	Name	Subject	Grade	Percentage
1	Nisha Patel	Math	A	92
2	Aarav Modi	Science	B+	85
3	Jiya Shah	English	A+	96

10) WAP to accept a float number and display with 2 decimals, 3 decimals, and width 10.

Input : 37.2567

Output :

2 decimals: 37.26

3 decimals: 37.257

Width 10: 37.26

```
In [22]: num = float(input("Enter a float number: "))

print(f"2 decimals: {num:.2f}")
print(f"3 decimals: {num:.3f}")
print(f"Width 10: {num:10.2f}")
```

```
Enter a float number: 37.2567
2 decimals: 37.26
3 decimals: 37.257
Width 10:      37.26
```

11) WAP to accept two integers and display sum, difference, and product using f-strings.

Input : 12 8

Output :

Sum = 20

Difference = 4

Product = 96

```
In [23]: a = int(input("Enter first number: "))
b = int(input("Enter second number: "))

print(f"Sum = {a + b}")
print(f"Difference = {a - b}")
print(f"Product = {a * b}")
```

```
Enter first number: 12
Enter second number: 8
Sum = 20
Difference = 4
Product = 96
```

12) WAP to accept date in dd mm yyyy format and display in multiple formats.

Input : 01 12 2025

Output :

01/12/2025

2025-12-01

```
In [26]: dd, mm, yyyy = input("Enter date (dd mm yyyy): ").split()

print(dd,mm,yyyy,sep = '/')
print/yyyy,mm,dd, sep = '-')
```

```
Enter date (dd mm yyyy): 01 12 2025
01/12/2025
2025-12-01
```

13) WAP to calculate area and perimeter of a circle.

```
In [27]: radius = float(input("Enter radius of the circle: "))
pi = 3.1416

area = pi * radius ** 2
perimeter = 2 * pi * radius

print(f"Area = {area:.2f}")
print(f"Perimeter = {perimeter:.2f}")
```

```
Enter radius of the circle: 10
Area = 314.16
Perimeter = 62.83
```

14) WAP to convert degree into Fahrenheit and vice versa.

```
In [28]: choice = input("Convert (D)egree to Fahrenheit or (F)ahrenheit to Degree? ").upper()

if choice == 'D':
    deg = float(input("Enter temperature in Celsius: "))
    fah = (deg * 9/5) + 32
    print(f"{deg}°C = {fah:.2f}°F")
elif choice == 'F':
    fah = float(input("Enter temperature in Fahrenheit: "))
    deg = (fah - 32) * 5/9
    print(f"{fah}°F = {deg:.2f}°C")
else:
    print("Invalid choice.")
```

```
Convert (D)egree to Fahrenheit or (F)ahrenheit to Degree? D
Enter temperature in Celsius: 45
45.0°C = 113.00°F
```

15) WAP to get the distance from user into kilometer, and convert it into meter, feet, inches and centimeter.

In [29]: `km = float(input("Enter distance in kilometers: "))`

```
meter = km * 1000
feet = km * 3280.84
inches = km * 39370.1
cm = km * 100000

print(f"Meters      = {meter:.2f}")
print(f"Feet       = {feet:.2f}")
print(f"Inches     = {inches:.2f}")
print(f"Centimeter = {cm:.2f}")
```

Enter distance in kilometers: 60

Meters = 60000.00

Feet = 196850.40

Inches = 2362206.00

Centimeter = 6000000.00

In []:

Python Programming - 2301CS404

Lab - 3

Jadeja Vishwrajsinh

24010101107

01) WAP to check whether the given number is positive or negative.

```
In [2]: num1 = int(input("Enter Numbers :"))
        if num1 > 0:
            print("Number is Positive.")
        else :
            print("Number is Negative")
```

Enter two Numbers : -5
Number is Negative

02) WAP to check whether the given number is odd or even.

```
In [5]: num1 = int(input("Enter Numbers :"))
        if num1 % 2 == 0:
            print("Number is Even.")
        else :
            print("Number is Odd")
```

Enter two Numbers : 4
Number is Even.

03) WAP to find out largest number from given two numbers using simple if and ternary operator.

```
In [7]: num1, num2 = input("Enter two Numbers :").split()
        num1 = int(num1)
        num2 = int(num2)

        if num1 > num2:
            res = num1;
        else:
            res = num2;
```

```
print(f"Largest number with using IF : {res}")

print(f"Largest number with using ternary : {num1}") if num1>num2 else print(f"L
```

Enter two Numbers : 4 10
 Largest number with using IF : 10
 Largest number with using ternary : 10

04) WAP to find out largest number from given three numbers.

```
In [10]: num1, num2, num3= input("Enter two Numbers :").split()
num1 = int(num1)
num2 = int(num2)
num3 = int(num3)

if num1>num2 and num1>num3:
    res = num1;
elif num2>num3:
    res = num2;
else:
    res = num3;
print(f"Largest number with using IF : {res}")

print(f"Largest number with using ternary : {num1}") if num1>num2 and num1>num3
```

Enter two Numbers : 8 2 5
 Largest number with using IF : 8
 Largest number with using ternary : 8

05) WAP to check whether the given year is leap year or not.

[If a year can be divisible by 4 but not divisible by 100 then it is leap year but if it is divisible by 400 then it is leap year]

```
In [15]: year = int(input("Enter Year :"))

if (year%4==0 and year%100!=0) or (year%400==0):
    print(f"{year} is leap year")
else:
    print(f"{year} is not leap year")
```

Enter Year : 2003
 2003 is not leap year

06) WAP in python to display the name of the day according to the number given by the user.

```
In [20]: day = int(input("Enter day:"))
day = day%7
match day:
    case 1:
        print("IT's Monday :-")
    case 2:
        print("Tuesday ~_~")
    case 3:
        print("Wensday ")
```



```

case 4:
    print("Thursday ")
case 5:
    print("Friday")
case 6:
    print("Saturday ")
case 7:
    print("Sunday :-)")
case _:
    print("False")

```

Enter day: 1
IT's Monday :-)

07) WAP to implement simple calculator which performs (add,sub,mul,div) of two no. based on user input.

```

In [ ]: num1,num2 = input("Enter a Two:").split()
num1 = int(num1)
num2 = int(num2)

day = int(input("Enter 1 for '+', 2 for '-', 3 for '*', 4 for '/':"))
match day:
    case 1:
        print("Ans = ",(num1+num2))
    case 2:
        print("Ans = ",(num1-num2))
    case 3:
        print("Ans = ",(num1*num2))
    case 4:
        print("Ans = ",(num1/num2))
    case _:
        print("False")

```

08) WAP to read marks of five subjects. Calculate percentage and print class accordingly.

Fail below 35

Pass Class between 35 to 45

Second Class

between 45 to 60

First Class between 60 to 70

Distinction if more than 70

```

In [29]: sub1, sub2, sub3, sub4, sub5, *s = map(int,input("Enetr your marks(5 subjects) :
print(type(sub1))
total = sub1+sub2+sub3+sub4+sub5

percentage = total / 5
if percentage <= 35:
    print("I am Sorry you are Fail")
elif percentage > 35 and percentage <= 45:

```

```

    print("Pass Class between 35 to 45 ")
elif percentage > 45 and percentage <= 60:
    print("Pass Second Class between 45 to 60")
elif percentage > 60 and percentage <= 70:
    print("Pass First Class between 60 to 70")
else:
    print("Invalid")

```

Enter your marks(5 subjects) : 40 50 60 70 80

<class 'int'>

Pass Second Class between 45 to 60

09) WAP to find the second largest number among three user input numbers.

```

In [33]: num1, num2, num3 = input("Enter two Numbers :").split()
num1 = int(num1)
num2 = int(num2)
num3 = int(num3)

if (num1>num2 and num1<num3) or (num1<num2 and num1>num3):
    res = num1;
elif num2<num3:
    res = num2;
else:
    res = num3;
print(f" Second Largest number with using IF : {res}")

```

Enter two Numbers : 2 8 5

Second Largest number with using IF : 5

10) WAP to calculate electricity bill based on following criteria. Which takes the unit from the user.

- First 1 to 50 units – Rs. 2.60/unit
- Next 50 to 100 units – Rs. 3.25/unit
- Next 100 to 200 units – Rs. 5.26/unit
- above 200 units – Rs. 8.45/unit

```

In [38]: bill = int(input("Enter Units :"))
total = float(0)

if bill <= 50:
    total = bill*2.60
elif bill <= 100:
    total = 50*2.60 + (bill-50)*3.25
elif bill <= 200:
    total = 50*2.60 + (50)*3.25 + (bill-100)*5.26
else :
    total = 50*2.60 + (50)*3.25 + (100)*5.26 + (bill-200)*8.45
print(f"Total Bill : {total}")

```

Enter Units : 234

Total Bill : 1105.8



Darshan
UNIVERSITY

Python Programming - 2301CS404

Lab - 4

Roll No.: 360

Name: Jadeja Vishwrajsinh

01) WAP to print 1 to 10.

```
In [60]: for i in range(1,11):  
         print(i,end=',')
```

1,2,3,4,5,6,7,8,9,10,

02) WAP to print 1 to n.

```
In [61]: num = int(input("Enter a Number :"))  
         for i in range(1,num+1):  
             print(i,end=',')
```

Enter a Number : 6

1,2,3,4,5,6,

03) WAP to print odd numbers between 1 to n.

```
In [62]: num = int(input("Enter a Number :"))  
         for i in range(1,num+1,2):  
             print(i,end=',')
```

Enter a Number : 6

1,3,5,

04) WAP to print numbers between two given numbers which is divisible by 2 but not divisible by 3.

```
In [63]: num1,num2 = map(int,input("Enter two numbers :").split())  
         min = num1 if(num1<num2) else num2  
         max = num1 if(num1>num2) else num2  
         for i in range(min,max+1):
```

```
if(i%2==0 and i%3!=0):  
    print(i,end=',')
```

Enter two numbers : 1 12
2,4,8,10,

05) WAP to print sum of 1 to n numbers.

```
In [15]: num = int(input("Enter a Number :"))  
sum = 0;  
for i in range(1,num+1):  
    sum = i+sum;  
print(f"Sum of 1 to {num} is: {sum}")  
sum1 = (num*(num+1))/2  
print(sum1)
```

Enter a Number : 5
Sum of 1 to 5 is: 15
15.0

06) WAP to print sum of series $1 + 4 + 9 + 16 + 25 + 36 + \dots n$.

```
In [26]: num = int(input("Enter a Number :"))  
odd = 3;  
temp = 1;  
for i in range(i,num+1):  
    print(temp,end=' ')  
    temp = odd+temp  
    odd = odd+2
```

Enter a Number : 9
1 4 9

07) WAP to print sum of series $1 - 2 + 3 - 4 + 5 - 6 + 7 \dots n$.

```
In [25]: num = int(input("Enter a number :"))  
if num>0:  
    print("1",end = ' ');  
for i in range(2,num+1):  
    if i%2 == 0:  
        print(f"- {i} ",end = '');  
    else:  
        print(f"+ {i} ",end = '');
```

Enter a number : 6
1 - 2 + 3 - 4 + 5 - 6

08) WAP to print Multiplication Table of the given number.

```
In [30]: num = int(input("Enter Table Number :"))  
for i in range(1,11):  
    print(f"{num:^5} X {i:^5} = {num*i:^5}")
```

Enter Table Number : 2

```

2   X   1   =   2
2   X   2   =   4
2   X   3   =   6
2   X   4   =   8
2   X   5   =  10
2   X   6   =  12
2   X   7   =  14
2   X   8   =  16
2   X   9   =  18
2   X  10   =  20

```

09) WAP to find Factorial of the given number.

```

In [38]: num = int(input("Enter a Number :"))
          sum = 1
          for i in range(num,1,-1):
              sum = sum*i
          print(f"{num}! = {sum}")

```

Enter a Number : 3
3! = 6

10) WAP to print GCD of given two numbers.

```

In [35]: num1,num2 = map(int,input("Enter two numbers:").split())
          min = num1 if num1<num2 else num2
          for i in range(min,0,-1):
              if num1%i==0 and num2%i==0:
                  print(f"{i} is a GCD of {num1} and {num2}")
                  break;

```

Enter two numbers: 6 8
2 is a GCD of 6 and 8

11) WAP to find Factors of the given number.

```

In [39]: num = int(input("Enter a Number :"))
          for i in range(1,i+1):
              if num%i==0:
                  print(f"{i}",end=',')

```

Enter a Number : 6
1,2,

12) WAP to find whether the given number is Prime or not.

```

In [44]: num = int(input("Enter a number:"))
          count = 0
          for i in range(2,num//2):
              if num % i == 0:
                  count = count+1
                  break;
          if count > 0:
              print(f"{num} is not a prime number.")
          else :
              print(f"{num} is a prime number.")

```

Enter a number: 11

11 is a prime number.

13) WAP to print sum of digits of given number.

```
In [47]: num = int(input("Enter a number:"))
temp = num
sum = 0
while temp>0:
    digit = temp%10
    temp = temp//10
    sum = sum + digit

print(sum)
```

Enter a number: 1234
10

14) WAP to check whether the given number is Palindrome or not.

```
In [49]: num = int(input("Enter a number:"))
temp = num
rev = 0
while temp>0:
    digit = temp%10
    temp = temp//10
    rev = rev*10 + digit
if rev == num :
    print(f"{num} is Palindrome.")
else :
    print(f"{num} is not Palindrome.")
```

Enter a number: 123
123 is not Palindrome.

15) WAP to check whether the given number is an Armstrong Number or not.

```
In [54]: num = int(input("Enter a number :"))
temp1 = num;
total_digit = 0
digit = 0
sum = 0

while temp1 > 0:
    temp1 = temp1//10
    total_digit = total_digit+1

temp2 = num
while temp2 > 0:
    digit = temp2%10
    temp2 = temp2//10
    pow = digit
    for i in range(1,total_digit):
        pow = pow*digit
    sum = sum+pow
if sum == num:
    print(f"{num} is Armstrong.")
```

```
else :  
    print(f"{num} is not Armstrong.")
```

Enter a number : 153
153 is Armstrong.

16) WAP to print all the perfect numbers between 1 to n.

```
In [59]: num = int(input("Enter a Number :"))  
for i in range(1,num+1):  
    sum = 0  
    for j in range(1,i):  
        if i%j==0:  
            sum = sum + j  
    if sum == i:  
        print(f"{i}",end=',')
```

Enter a Number : 28
6,28,

In []:



Darshan
UNIVERSITY

Python Programming - 2301CS404

Lab - 5

Roll No.: 360

Name: Jadeja Vishwrajsinh

1) Character Category Counter

Problem: Count uppercase, lowercase, digits, and special characters in a string.

Sample Inputs & Outputs:

- Input: Python3@Lab → Uppercase: 2, Lowercase: 7, Digits: 1, Special: 1
- Input: HELL0123 → Uppercase: 5, Lowercase: 0, Digits: 3, Special: 0
- Input: hi@#9 → Uppercase: 0, Lowercase: 2, Digits: 1, Special: 2

```
In [2]: str1 = input("Enter String:")
countup = 0
countlow = 0
countdigit = 0
countsep = 0
for i in str1:
    if i.isupper():
        countup += 1
    elif i.islower():
        countlow += 1
    elif i.isdigit():
        countdigit += 1
    else:
        countsep += 1

print(f"UpperCase : {countup}");
print(f"LowerCase : {countlow}");
print(f"Digit : {countdigit}");
print(f"Special : {countsep}");
```

Enter String: Python3@Lab

UpperCase : 2
LowerCase : 7
Digit : 1
Special : 1

2) Reverse Each Word

- Input: Python is Easy → nohtyP si ysaE
- Input: Hello World → olleH dlroW
- Input: Learn Python Fast → nraeL nohtyP tsaF

```
In [14]: s = input("Enter string: ")
output = ''

for word in s.split():
    output += word[::-1] + ' '

print(output.strip())
```

Enter string: Python is Easy
nohtyP si ysaE

3) Case Pattern Identifier

- PYTHON → Uppercase
- python → Lowercase
- Python Programming → Title Case

```
In [15]: s = input("Enter a String :")
if s.isupper():
    print("Uppercase")
elif s.islower():
    print("Lowercase")
elif s.istitle():
    print("Title Case")
else:
    print("Mixed Case")
```

Enter a String : PYTHON
Uppercase

4) First and Last Occurrence Finder

- programming , g → First:3 Last:10
- banana , a → First:1 Last:5
- hello , z → Not found

```
In [ ]: str1 = input()
char = input("Enetr char that u want to find: ")
First = str1.find(char)
Last = str1.rfind(char)
if(First != -1):
    print(f"First:{First} Last:{Last}");
else :
    print("Not found")
```

5) Word Frequency Counter (ignore the case)

- Python is easy and Python is powerful → python: 2
- Java is popular but java is verbose → java: 2
- C is fast → c:1

```
In [1]: str1 = input("Enter String :")
sub = input("Enter a substring That u want to count:");
print(str1.count(sub))
```

```
Enter String : Python is easy and Python is powerful
Enter a substring That u want to count: Python
2
```

6) Remove Extra Spaces

- Python is fun → Python is fun
- Hello World → Hello World
- NoExtraSpace → NoExtraSpace

```
In [4]: s = input ("Enter String")
output = ''
for i in s.split():
    output += i + ' '
print(output.rstrip())
```

```
Enter String   Python is fun
Python is fun
```

7) Prefix and Suffix Removal

- unhappy.txt → happy
- pretest.py → test
- unwanted.doc → wanted

```
In [5]: s = input("Enter String :")
pre = input("Enter a Prefix: ")
suf = input("Enter a Suffix")
s = s.removeprefix(pre)
s = s.removesuffix(suf)
print(s)
```

```
Enter String : unhappy.txt
Enter a Prefix: un
Enter a Suffix .txt
happy
```

8) Replace Vowels

- Programming → Pr*gr*mm*ng
- Education → *d*c*t**n
- Sky → Sky

```
In [7]: s = input("Enetr String :")
s = s.lower()
for i in s:
    if i in "aeiou":
        s = s.replace(i,"*")
print(s)
```

Enetr String : Programming
pr*gr*mm*ng

9) String Compression

- aaabbccccc → a3b2c4
- xxxyyyz → x3y3z1
- abcd → a1b1c1d1

```
In [20]: s = input("Enter String :")
count = 1
for i in range(0,len(s)-1):
    if s[i] == s[i+1]:
        count += 1
    else :
        print(f"{s[i]}{count}",end='')
        count = 1
print(f"{s[i+1]}{count}",end='')
```

Enter String : xxxyyyz
x3y3z1

10) Toggle Case (without using str.swapcase())

- PyThOn → pYtHoN
- HELlo → heLLo
- Python → pYTHON

```
In [9]: s = input("Enter a String :")
output = ''
for i in s:
    if i.isupper():
        output += i.lower()
    elif i.islower():
        output += i.upper()
print(output)
```

Enter a String : PyThOn
pYtHoN

11) Username Validation

valid username rules:

1. only alphabets and digits
2. does not start with a digit
3. length ≥ 6

- User123 → Valid
- 123User → Invalid
- Us@12 → Invalid

```
In [27]: s = input("Enter a valid Username :");
if s.isalnum() and len(s)>=6:
    print("Valid")
else :
    print("Invalid")
```

Enter a valid Username : Use12
Invalid

12) Palindrome Checker (ignore the case)

- Madam → Palindrome
- Level → Palindrome
- Python → Not Palindrome

```
In [13]: s = input("Enter a String :")
s = s.lower()
rev = s[::-1]
if s == rev:
    print("Palindrome.")
else :
    print("Not Palindrome.")
```

Enter a String : Madam
Palindrome.

13) Longest Word Finder

- Python programming is interesting → programming
- I love coding → coding
- Data structures and algorithms → structures

s = input("Enter String :")

output = " for i in s.split(): if len(output) < len(i): output = i print(output)

14) Case Conversion

- python programming
 - Upper: PYTHON PROGRAMMING
 - Lower: python programming
 - Title: Python Programming
 - Capitalize: Python programming

```
In [6]: s = input("Enter a string :")
print(f"Upper: {s.upper()}")
print(f"Lower: {s.lower()}")
```

```
print(f"Title: {s.title()}")
print(f"Capitalize: {s.capitalize()}")
```

Enter a string : Pyhton Programming
Upper:PYHTON PROGRAMMING
Lower:pyhton programming
Title:Pyhton Programming
Capitalize:Pyhton programming

15) Custom Split

- apple,banana,grape
- 10|20|30
- A-B-C-D

```
In [24]: char = input ("Enetr your custom split special char:")
s = input("Enetr string :")
output = ''
for i in s.split():
    output = output+i+char
print(output[0:len(output)-1])
```

Enetr your custom split special char: ,
Enetr string : apple banana grape
apple,banana,grape

In []:

Python Programming - 2301CS404

Lab - 6

Roll No. : 360

Name : Jadeja Vishwrajsinh

01) WAP to find sum of all the elements in a List.

```
In [2]: l1 = [1,2,3,4,5]
sum = 0
for i in l1:
    sum += i
print(f"Sum of all element in list = {sum}")
```

Sum of all element in list = 15

02) WAP to find largest element in a List.

```
In [5]: l1 = [2,3,5,1,4]
max1 = l1[0]
for i in l1:
    if max1 < i:
        max1 = i
print(f"Largest Element : {max1}")
```

Largest Element : 5

03) WAP to find the length of a List.

```
In [6]: l1 = [1,2,3,4,5,6]
print(len(l1))
```

6

04) WAP to interchange first and last elements in a list.

```
In [8]: l1 = [1,2,3,4,5,6]
temp = l1[0]
```

```
l1[0] = l1[len(l1)-1]
l1[len(l1)-1] = temp
print(l1)
```

[6, 2, 3, 4, 5, 1]

05) WAP to split the List into two parts and append the first part to the end.

```
In [16]: l1 = [1,2,3,4,5,6]
l2 = []
i = 0
while i <= len(l1)//2:
    l2.append(l1.pop(0))
    i += 1
l1.extend(l2)
print(l1)
```

[4, 5, 6, 1, 2, 3]

06) WAP to interchange the elements on two positions entered by a user.

```
In [18]: l1 = [1,2,3,4,5,6]
index1,index2 = map(int,input("Enter two elements positions : ").split())
temp = l1[index1]
l1[index1] = l1[index2]
l1[index2] = temp
print(l1)
```

Enter two elements positions : 2 4

[1, 2, 5, 4, 3, 6]

07) WAP to reverse the list entered by user.

```
In [22]: l1 = [1,2,3,4,5,6]
i = 0
l1.reverse()
print(l1)
```

[6, 5, 4, 3, 2, 1]

08) WAP to print even numbers in a list.

```
In [27]: l1 = [1,2,3,4,5,6]
evenCount = 0
for i in l1:
    if i % 2 == 0:
        evenCount += 1
print("Number of Even element in list:",evenCount)
```

Number of Even element in list : 3

09) WAP to count unique items in a list.

```
In [12]: l1 = list(map(int, input("Enter a List:").split()))
countUnique = 0
```

```

for i in range(len(l1)):
    flage = True
    for j in range(1,len(l1)):
        if l1[i] == l1[j]:
            if i != j:
                flage = False
                break;
    if flage:
        countUnique += 1
print(countUnique)

```

Enter a List: 10 20 30 10
0 10
1 20
2 30
3 10
3

10) WAP to copy a list.

```

In [29]: l1 = [1,2,3,4,5,6,7]
         l2 = l1.copy()
         print(l2)

```

[1, 2, 3, 4, 5, 6, 7]

11) WAP to print all odd numbers in a given range.

```

In [30]: ans = []
         num1,num2 = map(int,input("Enter a Range :").split())
         for i in range(num1,num2+1):
             if i % 2 != 0:
                 ans.append(i)
         print(ans)

```

Enter a Range : 1 6
[1, 3, 5]

12) WAP to count occurrences of an element in a list.

```

In [31]: l1 = [1,23,3,4,5,6,1,2,3,1]
         print(l1.count(1))

```

3

13) WAP to find second largest number in a list.

```

In [40]: l1 = [1,4,2,5,6,3]
         max1 = l1[0]
         secondMax = 0
         for i in l1:
             if max1 < i:
                 secondMax = max1
                 max1 = i
             elif secondMax < i and i != max1:
                 secondMax = i
         print("Second Largest Element from list is:",secondMax)

```


Second Largest Element from list is: 5

14) WAP to extract elements with frequency greater than K.

```
In [18]: l1 = list(map(int, input("Enter a List:").split()))
k = int(input("k :"))
ans = []

for i in range(len(l1)):
    count = 1
    for j in range(0, len(l1)):
        if l1[i] == l1[j] and i != j:
            count += 1
    if count > k:
        ans.append(l1[i])
res = []
for i in ans:
    if i not in res:
        res.append(i)
print(res)
```

```
Enter a List: 1 2 3 1 2 3 1 1 1
k : 4
[1]
```

15) WAP to create a list of squared numbers from 0 to 9 with and without using List Comprehension.

```
In [26]: l1 = [i for i in range(10)]
l2 = []
for i in l1:
    l2.append(i**2)
print(l2)
```

```
[0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
```

16) WAP to create a new list (fruit whose name starts with 'b') from the list of fruits given by user.

```
In [ ]: l1 = list(input("Enter a List:").split())
ans = []
for i in l1:
    i.lower()
    if i[0]=='b':
        ans.append(i)
print(ans)
```

17) WAP to create a list of common elements from given two lists.

```
In [33]: l1 = list(map(int, input("Enter List1:").split()))
l1.sort()
l2 = list(map(int, input("Enter List1:").split()))
l2.sort()
ans = []
i = 0
```

```
j = 0
print(l1)
print(l2)
while i<len(l1) and j<len(l2):
    if l1[i] == l2[j]:
        ans.append(l1[i])
        i += 1
        j += 1
    elif l1[i] < l2[j]:
        i+=1
    else :
        j+=1

print(ans)
```

Enter List1: 3 2 1 1

Enter List1: 0 1 1 2 4 1 1 2 2 3 3

[1, 1, 2, 3]

[0, 1, 1, 1, 1, 2, 2, 2, 3, 3, 4]

[1, 1, 2, 3]

In []:



Darshan
UNIVERSITY

Python Programming - 2301CS404

Lab - 7

Roll No : 360

Name :Jadeja Vishwrajsinh

01) WAP to find sum of tuple elements.

Sample Input:

T = (10, 20, 30, 40)

Sample Output:

Sum of tuple elements = 100

```
In [1]: T = tuple(map(int,input("Enter a Numbers").split()))
total = 0
for i in T:
    total += i
print("Sum of All element in the tuple:",total)
```

Sum of All element in the tuple: 100

02) WAP to find Maximum and Minimum K elements in a given tuple.

Sample Input:

T = (7, 2, 9, 4, 1, 5)
K = 2

Sample Output:

Minimum 2 elements: (1, 2)
Maximum 2 elements: (9, 7)

```
In [1]: T = tuple(map(int,input("Enter a Numbers").split()))

k = int(input("Entre a value of k:"))
maximum = []
minimum = []

temp1 = list(T)
temp2 = list(T)

while k > 0:
    max_value = max(temp1)
    min_value = min(temp2)
    temp1.remove(max_value)
    temp2.remove(min_value)
    minimum.append(min_value)
    maximum.append(max_value)
    k -= 1
print(f"Minimum {k} element {minimum}")
print(f"Minimum {k} element {maximum}")
```

Enter a Numbers 7 2 9 4 1 5
Entre a value of k: 2
Minimum 0 element [1, 2]
Minimum 0 element [9, 7]

03) WAP to find tuples which have all elements divisible by K from a list of tuples.

Sample Input:

```
tuple_list = [(2, 4, 6), (3, 6, 9), (4, 8, 12), (5, 10, 15)]
K = 2
```

Sample Output:

Tuples divisible by 2 : [(2, 4, 6), (4, 8, 12)]

```
In [3]: len1 = int(input("Enter a lenth of list:"))
tuple_list = []
for i in range(0,len1):
    temp = tuple(map(int,input().split()))
    tuple_list.append(temp)
k = int(input("Enter a value of k:"))
ans = []
for i in tuple_list:
    flage = True
    for j in i:
        if j % k != 0:
            flage = False
    if flage:
        ans.append(i)
print(f"Tuple divisible by {k} : {ans}")
```

```
Enter a lenth of list: 4
2 4 6
3 6 9
4 8 12
5 10 15
Enter a value of k: 2
Tuple divisible by 2 : [(2, 4, 6), (4, 8, 12)]
```

04) WAP to create a list of tuples from given list having number and its cube in each tuple.

Sample Input:

```
numbers = [1, 2, 3, 4, 5]
```

Sample Output:

```
List of tuples with number and its cube: [(1, 1), (2, 8), (3, 27), (4, 64), (5, 125)]
```

In []:

05) WAP to find tuples with all positive elements from the given list of tuples.

Sample Input:

```
tuple_list = [(1, 2, 3), (-1, 2, 3), (4, 5, 6), (0, 1, 2), (-3, -4, -5)]
```

Sample Output:

```
Tuples with all positive elements: [(1, 2, 3), (4, 5, 6)]
```

```
In [6]: len1 = int(input("Enter a lenth of list:"))
tuple_list = []
for i in range(0,len1):
    temp = tuple(map(int,input().split()))
    tuple_list.append(temp)
ans = []
for i in tuple_list:
    flage = True
    for j in i:
        if j <= 0:
            flage = False
    if flage:
        ans.append(i)
print(f"Tuples with all positive elements:: {ans}")
```

```
Enter a lenth of list: 3
1 2 3
-1 2 3
0 1 2
```

Tuples with all positive elements:: [(1, 2, 3)]

06) WAP to remove tuples of length K.

Sample Input:

```
tuple_list = [(1, 2), (3, 4, 5), (6,), (7, 8, 9), (10, 11)]
K = 2
```

Sample Output:

```
List after removing tuples of length 2 : [(3, 4, 5), (6,), (7,
8, 9)]
```

```
In [21]: tuple_list = [(1, 2), (3, 4, 5), (6,), (7, 8, 9), (10, 11)]
k = int(input("Enter a value of k:"))
ans = []
for i in tuple_list:
    count = 0
    for j in i:
        count += 1
    if count != k:
        ans.append(i)
print(f"List after removing tuples of length 2 : {ans}")
```

Enter a value of k: 2

List after removing tuples of length 2 : [(3, 4, 5), (6,), (7, 8, 9)]

07) WAP to remove all occurrences of a given element from a tuple.

Sample Input:

```
T = (3, 5, 3, 7, 3, 9)
x = 3
```

Sample Output:

```
(5, 7, 9)
```

```
In [24]: T = (3, 5, 3, 7, 3, 9)
x = int(input("Enter a value of x:"))
ans = []
for i in T:
    if i != x:
        ans.append(i)
print(ans)
```

Enter a value of x: 3

[5, 7, 9]

08) WAP to remove duplicates from tuple.

Sample Input:

T = (3, 5, 3, 7, 3, 9)

Sample Output:

(3, 5, 7, 9)

```
In [27]: T = (3, 5, 3, 7, 3, 9)
ans = []
for i in T:
    if i not in ans:
        ans.append(i)
print(ans)
```

[3, 5, 7, 9]

09) WAP to multiply adjacent elements of a tuple and print that resultant tuple.

Sample Input:

t = (2, 3, 4, 5)

Sample Output:

Resultant tuple after multiplying adjacent elements: (6, 12, 20)

```
In [32]: t = (2, 3, 4, 5)
ans = []

for i in range(len(t)-1):
    ans.append(t[i] * t[i+1])

print(ans)
```

[6, 12, 20]

10) WAP to test if the given tuple is distinct or not.

Sample Input:

tuple1 = (1, 2, 3, 4)
tuple2 = (1, 2, 2, 3)

Sample Output:

(1, 2, 3, 4) is distinct? -> True
(1, 2, 2, 3) is distinct? -> False

```
In [34]: tuple1 = (1, 2, 3, 4)
tuple2 = (1, 2, 2, 3)
ans1 = []
ans2 = []
flage1 = True
flage2 = True
```

```

for i in tuple1:
    if i in ans1:
        flage1 = False
        break
    ans1.append(i)

for i in tuple2:
    if i in ans2:
        flage2 = False
        break
    ans2.append(i)

print(f"{tuple1} is distinct? -> {flage1}")
print(f"{tuple2} is distinct? -> {flage2}")

```

(1, 2, 3, 4) is distinct? -> True
 (1, 2, 2, 3) is distinct? -> False

11) WAP to rotate the elements of a tuple to the right by `k` positions.

Sample Input:

```

T = (10, 20, 30, 40, 50)
k = 2

```

Sample Output:

```

(40, 50, 10, 20, 30)

```

```

In [13]: T = tuple(map(int,input("Enter a Numbers").split()))
k = int(input("Enter a value of k:"))
T_list = list(T)
i=0

for j in range(len(T_list)):
    if i < k:
        T_list.append(T_list.pop(j))
        i+=1
T = tuple(T_list)
print(T)

```

```

Enter a Numbers 10 20 30 40 50
Enter a value of k: 2
(20, 40, 50, 10, 30)

```

12) WAP to merge two tuples of equal length alternately.

Sample Input:

```

T1 = (1, 3, 5)
T2 = (2, 4, 6)

```

Sample Output:

```

(1, 2, 3, 4, 5, 6)

```



```
In [2]: T1 = tuple(map(int,input("Enter a Numbers").split()))
        T2 = tuple(map(int,input("Enter a Numbers").split()))
        ans = tuple()
        for i,j in zip(T1,T2):
            ans += (i,j)
        print(ans)
```

Enter a Numbers 1 3 5

Enter a Numbers 2 4 6

(1, 2, 3, 4, 5, 6)

13) WAP to create a tuple of squares of elements that are even and greater than 10.

Sample Input:

T = (4, 12, 7, 18, 10)

Sample Output:

(144, 324)

```
In [ ]: T1 = tuple(map(int,input("Enter a Numbers").split()))
        ans = tuple()
        for i in T1:
            if i > 10 and i % 2 == 0:
                ans += (i**2,)
        print(ans)
```

14) WAP to create (index, value) pairs for prime numbers.

Sample Input:

T = (4, 7, 9, 11, 15)

Sample Output:

((1, 7), (3, 11))

```
In [3]: T = (4, 7, 9, 11, 15)

        result = tuple((idx, val) for idx, val in enumerate(T) if is_prime(val))

        print(result)

        def is_prime(n):
            if n < 2:
                return False
            for i in range(2, int(n**0.5) + 1):
                if n % i == 0:
                    return False
            return True
```

((1, 7), (3, 11))

15) WAP to split a tuple into even-index and odd-index tuples.

Sample Input:

```
T = ('a', 'b', 'c', 'd', 'e')
```

Sample Input:

```
even = ('a', 'c', 'e')
odd = ('b', 'd')
```

```
In [5]: user_input = input("Enter elements separated by space: ").split()
T = tuple(user_input)

even = T[::2]

odd = T[1::2]

print(f"Original Tuple: {T}")
print(f"even = {even}")
print(f"odd = {odd}")
```

```
Enter elements separated by space: s o h a m
Original Tuple: ('s', 'o', 'h', 'a', 'm')
even = ('s', 'h', 'm')
odd = ('o', 'a')
```

16) WAP to compute column-wise sum from a tuple of lists.

Sample Input:

```
T = ([1, 2, 3], [4, 5, 6], [7, 8, 9])
```

Sample Output:

```
12 15 18
```

```
In [7]: num_lists = int(input("Enter the number of lists: "))

temp_list = []
for i in range(num_lists):
    row_input = input(f"Enter elements for list {i+1} (separated by space): ")

    row = [int(x) for x in row_input.split()]
    temp_list.append(row)

T = tuple(temp_list)
column_sums = [sum(column) for column in zip(*T)]

print("\nColumn-wise sums:")
print(*(column_sums))
```

```
Enter the number of lists: 3
Enter elements for list 1 (separated by space): 1 2 3
Enter elements for list 2 (separated by space): 4 5 6
Enter elements for list 3 (separated by space): 7 8 9
Column-wise sums:
12 15 18
```

In []:



Darshan
UNIVERSITY

Python Programming - 2301CS404

Lab - 8

Roll No.: 360

Name: Jadeja Vishwrajsinh/b>

01) WAP to find their union, intersection, and difference of the given two sets.

Sample Input:

A = {1, 2, 3, 4}
B = {3, 4, 5, 6}

Sample Output:

Union: {1, 2, 3, 4, 5, 6}
Intersection: {3, 4}
Difference (A-B): {1, 2}

```
In [2]: n1 = int(input("Enter number of elements in set A: "))
A = []
for i in range(n1):
    A.append(int(input()))

n2 = int(input("Enter number of elements in set B: "))
B = []
for i in range(n2):
    B.append(int(input()))

setA = set(A)
setB = set(B)

print("Union:", setA | setB)
print("Intersection:", setA & setB)
print("Difference (A-B):", setA - setB)
```

Enter number of elements in set A: 5

1
2
3
4
5

Enter number of elements in set B: 2

1
2

Union: {1, 2, 3, 4, 5}

Intersection: {1, 2}

Difference (A-B): {3, 4, 5}

02) WAP to Find elements common to all three given sets.

Sample Input:

A = {1, 2, 3, 4}
B = {2, 3, 5}
C = {2, 3, 6}

Sample Output:

{2, 3}

```
In [7]: n1 = int(input("Enter number of elements in set A: "))
A = []
for i in range(n1):
    A.append(int(input("Enter element of set A: ")))

n2 = int(input("Enter number of elements in set B: "))
B = []
for i in range(n2):
    B.append(int(input("Enter element of set B: ")))

n3 = int(input("Enter number of elements in set C: "))
C = []
for i in range(n3):
    C.append(int(input("Enter element of set C: ")))
```

```

setA = set(A)
setB = set(B)
setC = set(C)

print(setA & setB & setC)

```

```

Enter number of elements in set A: 4
Enter element of set A: 1
Enter element of set A: 2
Enter element of set A: 3
Enter element of set A: 4
Enter number of elements in set B: 3
Enter element of set B: 2
Enter element of set B: 3
Enter element of set B: 5
Enter number of elements in set C: 3
Enter element of set C: 2
Enter element of set C: 3
Enter element of set C: 6
{2, 3}

```

03) WAP to remove duplicate elements from a list while preserving unique values.

Sample Input:

```
[10, 20, 10, 30, 20, 40]
```

Sample Output:

```
{10, 20, 30, 40}
```

```

In [9]: n = int(input("Enter number of elements: "))
        lst = []
        for i in range(n):
            lst.append(int(input("Enter element: ")))

        unique = set(lst)
        print(unique)

```

```

Enter number of elements: 4
Enter element: 1
Enter element: 1
Enter element: 2
Enter element: 2
{1, 2}

```

04) WAP to count how many elements are common between two sets.

Sample Input:

```

A = {1, 2, 3, 4}
B = {3, 4, 5, 6}

```

Sample Output:

2

```
In [10]: n1 = int(input("Enter number of elements in set A: "))
A = []
for i in range(n1):
    A.append(int(input("Enter element of set A: ")))

n2 = int(input("Enter number of elements in set B: "))
B = []
for i in range(n2):
    B.append(int(input("Enter element of set B: ")))

setA = set(A)
setB = set(B)

print(len(setA & setB))
```

```
Enter number of elements in set A: 4
Enter element of set A: 1
Enter element of set A: 2
Enter element of set A: 3
Enter element of set A: 4
Enter number of elements in set B: 5
Enter element of set B: 3
Enter element of set B: 4
Enter element of set B: 5
Enter element of set B: 6
Enter element of set B: 7
2
```

05) WAP to Merge two dictionaries. If a key exists in both dictionaries, add their values.

Sample Input:

```
Dict1 = {1: 10, 2: 20, 3: 30}
Dict2 = {2: 40, 3: 50, 4: 60}
```

Sample Output:

```
{1: 10, 2: 60, 3: 80, 4: 60}
```

```
In [11]: n1 = int(input("Enter number of key-value pairs in Dict1: "))
dict1 = {}
for i in range(n1):
    key = int(input("Enter key for Dict1: "))
    value = int(input("Enter value for Dict1: "))
    dict1[key] = value

n2 = int(input("Enter number of key-value pairs in Dict2: "))
dict2 = {}
for i in range(n2):
    key = int(input("Enter key for Dict2: "))
    value = int(input("Enter value for Dict2: "))
```

```

dict2[key] = value

for key in dict2:
    if key in dict1:
        dict1[key] = dict1[key] + dict2[key]
    else:
        dict1[key] = dict2[key]

print(dict1)

```

Enter number of key-value pairs in Dict1: 3
Enter key for Dict1: 1
Enter value for Dict1: 10
Enter key for Dict1: 2
Enter value for Dict1: 20
Enter key for Dict1: 3
Enter value for Dict1: 30
Enter number of key-value pairs in Dict2: 3
Enter key for Dict2: 2
Enter value for Dict2: 40
Enter key for Dict2: 3
Enter value for Dict2: 50
Enter key for Dict2: 4
Enter value for Dict2: 60
{1: 10, 2: 60, 3: 80, 4: 60}

06) WAP to Sort a dictionary in ascending order of its values.

Hint: in sorted(), use udf for key parameter

Sample Input:

```
{'Math': 70, 'Physics': 85, 'Chemistry': 60}
```

Sample Output:

```
{'Chemistry': 60, 'Math': 70, 'Physics': 85}
```

```

In [ ]: n = int(input("Enter number of key-value pairs: "))
d = {}
for i in range(n):
    key = input("Enter key: ")
    value = int(input("Enter value: "))
    d[key] = value

def get_value(item):    # used to extract the dictionary value so sorting happens
    return item[1]

items = list(d.items())    # produces (key, value) pairs.
items.sort(key=get_value) # before sorting each item, pass it to the function ge

sorted_dict = {}
for k, v in items:
    sorted_dict[k] = v

```



```
print(sorted_dict)
```

07) WAP to find the keys having maximum and minimum values in a dictionary.

Sample Input:

```
{'A': 50, 'B': 90, 'C': 30}
```

Sample Output:

```
Max Value Key: B  
Min Value Key: C
```

```
In [12]: n = int(input("Enter number of key-value pairs: "))  
d = {}  
for i in range(n):  
    key = input("Enter key: ")  
    value = int(input("Enter value: "))  
    d[key] = value  
  
max_value = max(d.values())  
min_value = min(d.values())  
  
for k in d:  
    if d[k] == max_value:  
        print("Max Value Key:", k)  
        break  
  
for k in d:  
    if d[k] == min_value:  
        print("Min Value Key:", k)  
        break
```

```
Enter number of key-value pairs: 3  
Enter key: A  
Enter value: 50  
Enter key: B  
Enter value: 90  
Enter key: C  
Enter value: 30  
Max Value Key: B  
Min Value Key: C
```

08) WAP to create a dictionary with numbers from 1 to N as keys and their squares as values.

Sample Input:

```
N = 5
```

Sample Output:

```
{1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
```

```
In [16]: n = int(input("Enter value of N: "))
d = {}

for i in range(1, n + 1):
    d[i] = i * i

print(d)
```

Enter value of N: 4
{1: 1, 2: 4, 3: 9, 4: 16}

09) WAP to count the frequency of each character in a string using a dictionary.

Sample Input:

```
programming
```

Sample Output:

```
{'p': 1, 'r': 2, 'o': 1, 'g': 2, 'a': 1, 'm': 2, 'i': 1, 'n': 1}
```

```
In [22]: s = input("Enter a string: ")
d = {}

for ch in s:
    if ch in d:
        d[ch] = d[ch] + 1
    else:
        d[ch] = 1

print(d)
```

Enter a string: RakshitOP
{'R': 1, 'a': 1, 'k': 1, 's': 1, 'h': 1, 'i': 1, 't': 1, 'O': 1, 'P': 1}

10) WAP to swap keys and values of a dictionary.

Sample Input:

```
{1: 'A', 2: 'B', 3: 'C'}
```

Sample Output:

```
{'A': 1, 'B': 2, 'C': 3}
```

```
In [24]: n = int(input("Enter number of key-value pairs: "))
d = {}
for i in range(n):
    key = int(input("Enter key: "))
    value = input("Enter value: ")
    d[key] = value
```

```
swapped = {}

for k in d:
    swapped[d[k]] = k

print(swapped)
```

Enter number of key-value pairs: 1
Enter key: 2
Enter value: 3
{'3': 2}

11) WAP to find the student with the highest total marks, given student names and their marks.

Sample Input:

```
{'Amit': [70, 80, 90], 'Riya': [85, 75, 80], 'Neha': [60, 70, 65]}
```

Sample Output:

Topper: Amit

```
In [32]: n = int(input("Enter number of students: "))
m = int(input("Enter number of subjects: "))

students = {}

for i in range(n):
    name = input("Enter student name: ")
    marks = []
    for j in range(m):
        marks.append(int(input("Enter mark: ")))
    students[name] = marks

topper = None
highest_total = None

for name in students:
    total = 0
    for mark in students[name]:
        total = total + mark
    if highest_total is None or total > highest_total:
        highest_total = total
        topper = name

print("Topper:", topper)
```

Enter number of students: 2
Enter number of subjects: 1
Enter student name: Rakshit1
Enter mark: 10
Enter student name: Rakshit2
Enter mark: 10
Topper: Rakshit1

12) WAP to extract keys whose values are even numbers.

Sample Input:

{1: 11, 2: 20, 3: 15, 4: 40}

Sample Output:

[2, 4]

```
In [34]: n = int(input("Enter number of key-value pairs: "))
d = {}

for i in range(n):
    key = int(input("Enter key: "))
    value = int(input("Enter value: "))
    d[key] = value

result = []

for k in d:
    if d[k] % 2 == 0:
        result.append(k)

print(result)
```

```
Enter number of key-value pairs: 2
Enter key: 1
Enter value: 10
Enter key: 2
Enter value: 17
[1]
```

13) WAP to Find common keys between two dictionaries.

Sample Input:

Dict1 = {1: 'A', 2: 'B', 3: 'C'}
Dict2 = {2: 'X', 3: 'Y', 4: 'Z'}

Sample Output:

[2, 3]

```
In [35]: n1 = int(input("Enter number of key-value pairs in Dict1: "))
d1 = {}
for i in range(n1):
    key = int(input("Enter key for Dict1: "))
    value = input("Enter value for Dict1: ")
    d1[key] = value

n2 = int(input("Enter number of key-value pairs in Dict2: "))
d2 = {}
for i in range(n2):
    key = int(input("Enter key for Dict2: "))
    value = input("Enter value for Dict2: ")
```

```

    d2[key] = value

common_keys = []

for k in d1:
    if k in d2:
        common_keys.append(k)

print(common_keys)

```

```

Enter number of key-value pairs in Dict1: 2
Enter key for Dict1: 1
Enter value for Dict1: A
Enter key for Dict1: 2
Enter value for Dict1: B
Enter number of key-value pairs in Dict2: 2
Enter key for Dict2: 2
Enter value for Dict2: X
Enter key for Dict2: 3
Enter value for Dict2: Y
[2]

```

14) WAP to handle missing keys in dictionaries.

Sample Input - 1:

```

dict1 = {'a': 5, 'c': 8, 'e': 2}
key = 'c'

```

Sample Output - 1:

```

8

```

Sample Input - 2:

```

dict1 = {'a': 5, 'c': 8, 'e': 2}
key = 'd'

```

Sample Output - 2:

```

Key Not Found

```

```

In [37]: n = int(input("Enter number of key-value pairs: "))
d = {}

for i in range(n):
    key = input("Enter key: ")
    value = int(input("Enter value: "))
    d[key] = value

search_key = input("Enter key to search: ")

if search_key in d:
    print(d[search_key])
else:
    print("Key Not Found")

```

```
Enter number of key-value pairs: 2
Enter key: a
Enter value: 2
Enter key: b
Enter value: 3
Enter key to search: a
2
```

15) WAP to create a dictionary using dictionary comprehension from two lists containing student names and marks. Include only those students in the dictionary whose marks are greater than or equal to 75.

Sample Input:

```
students = ['Amit', 'Riya', 'Neha', 'Karan']
marks = [78, 72, 85, 60]
```

Sample Output:

```
{'Amit': 78, 'Neha': 85}
```

```
In [38]: n = int(input("Enter number of students: "))

students = []
marks = []

for i in range(n):
    students.append(input("Enter student name: "))

for i in range(n):
    marks.append(int(input("Enter marks: ")))

result = {students[i]: marks[i] for i in range(n) if marks[i] >= 75}

print(result)
```

```
Enter number of students: 2
Enter student name: Rakshit1
Enter student name: Rakshit2
Enter marks: 67
Enter marks: 69
{}
```

```
In [ ]:
```



Darshan
UNIVERSITY

Python Programming - 24010101107

Lab - 9

Roll No : 360

Name : Jadeja Vishwrajsinh

01) Write a function to calculate BMI given mass and height.
(BMI = mass/h**2)

```
In [3]: def BMI(m,h):  
        return m/(h**2)  
  
        m = float(input("Enter value of mass: "))  
        h = float(input("Enter value of height: "))  
  
        print(f"BMI = {BMI(m,h)}")
```

BMI = 5.0

02) Write a function that add first n numbers.

```
In [6]: def add(n):  
        return n*(n+1)/2  
  
        n = int(input("Enter value of n: "))  
        print(f"Sum = {add(n)}")
```

Sum = 55.0

03) Write a function that returns 1 if the given number is
Prime or 0 otherwise.

```
In [12]: def is_prime(n):  
        if n <= 1:  
            return 0
```

```

    for i in range(2, int(n ** 0.5) + 1):
        if n % i == 0:
            return 0
    return 1

n = int(input("Enter a number: "))

print(is_prime(n))

```

0

04) Write a function that returns the list of Prime numbers between given two numbers.

```

In [13]: def primes_between(start, end):
    primes = []
    for num in range(max(2, start), end + 1):
        is_prime = True
        for i in range(2, int(num ** 0.5) + 1):
            if num % i == 0:
                is_prime = False
                break
        if is_prime:
            primes.append(num)
    return primes

start = int(input("Enter the starting number: "))
end = int(input("Enter the ending number: "))

print("Prime numbers between", start, "and", end, "are:")
print(primes_between(start, end))

```

Prime numbers between 1 and 50 are:
[2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47]

05) Write a function that returns True if the given string is Palindrome or False otherwise.

```

In [17]: def is_palindrome(s):
    return s == s[::-1]

s = input()
print(is_palindrome(s))

```

True

06) Write a function that returns the sum of all the elements of the list.

```

In [ ]: def sum_of_list(lst):
    return sum(lst)

lst = list(map(int, input().split()))
print(sum_of_list(lst))

```


07) Write a function to calculate the sum of the first element of each tuples inside the list.

```
In [ ]: def sum_first_elements(tuples_list):  
        return sum(t[0] for t in tuples_list)  
  
n = int(input())  
tuples_list = [tuple(map(int, input().split())) for _ in range(n)]  
print(sum_first_elements(tuples_list))
```

08) Write a recursive function to find nth term of Fibonacci Series.

```
In [35]: def fibonacci(n):  
        if n <= 1:  
            return n  
        return fibonacci(n - 1) + fibonacci(n - 2)  
  
n = int(input())  
print(fibonacci(n))
```

8

09) Write a function to get the name of the student based on the given rollno.

Example: Given dict1 = {101:'Ajay', 102:'Rahul', 103:'Jay', 104:'Pooja'} find name of student whose rollno = 103

```
In [26]: def get_student_name(d, rollno):  
        return d.get(rollno)  
  
n = int(input())  
d = {}  
for _ in range(n):  
    k, v = input().split()  
    d[int(k)] = v  
rollno = int(input())  
print(get_student_name(d, rollno))
```

Rakshit

10) Write a function to get the sum of the scores ending with zero.

Example : scores = [200, 456, 300, 100, 234, 678]

Ans = 200 + 300 + 100 = 600

```
In [27]: def scoress(scores):  
        return sum(x for x in scores if x % 10 == 0)  
  
scores = list(map(int, input().split()))  
print(scoress(scores))
```

11) Write a function to invert a given Dictionary.

hint: keys to values & values to keys

Before : {'a': 10, 'b':20, 'c':30, 'd':40}

After : {10:'a', 20:'b', 30:'c', 40:'d'}

```
In [28]: def invert_dict(d):
          return {v: k for k, v in d.items()}

          print(invert_dict({'a': 10, 'b': 20, 'c': 30, 'd': 40}))

{10: 'a', 20: 'b', 30: 'c', 40: 'd'}
```

12) Write a function that returns the number of uppercase and lowercase letters in the given string.

example : Input : s1 = AbcDEfgh ,Ouputput : no_upper = 3, no_lower = 5

```
In [29]: def count_up_low(s):
          up = sum(1 for ch in s if ch.isupper())
          low = sum(1 for ch in s if ch.islower())
          return up, low

          print(count_up_low("AbcDEfgh"))

(3, 5)
```

13) Write a lambda function to get smallest number from the given two numbers.

```
In [30]: small = lambda a, b: a if a < b else b
          print(small(10, 4))

4
```

14) For the given list of names of students, extract the names having more that 7 characters. Use filter().

```
In [31]: names = ["Ruchit", "Aryan", "Soham", "Manas", "Rakshit"]
          long_names = list(filter(lambda x: len(x) > 7, names))
          print(long_names)

['Alexander', 'Christina', 'Jonathan']
```

15) For the given list of names of students, convert the first letter of all the names into uppercase. use map().

```
In [32]: names = ["ram", "shyam", "ravi"]
          cap_names = list(map(lambda x: x.capitalize(), names))
          print(cap_names)

['Ram', 'Shyam', 'Ravi']
```

16) Write udfs to call the functions with following types of arguments:

1. Positional Arguments
2. Keyword Arguments
3. Default Arguments
4. Variable Length Positional(*args) & variable length Keyword Arguments (**kwargs)
5. Keyword-Only & Positional Only Arguments

```
In [ ]: # Positional Args
def pos(a, b):
    return a + b

print(pos(5, 10))

# Keyword Args
def kw(a=0, b=0):
    return a + b

print(kw(b=5, a=3))

# Default Args
def default(a=10, b=20):
    return a + b

print(default())

# Variable Length
def var(*args, **kwargs):
    print(args, kwargs)

var(1,2,3, x=10, y=20)

# Keyword-only & Positional-only
def mix(a, /, b, *, c):
    print(a, b, c)

mix(1, b=2, c=3)
```



Darshan
UNIVERSITY

Python Programming - 24010101107

Lab - 10

Roll No : 360

Name : Jadeja Vishwrajsinh

01) WAP to read and display the contents of a text file. (also try to open the file in some other directory)

- in the form of a string
- line by line
- in the form of a list

```
In [5]: # 1) Read file as a string
f1 = open("file1.txt", "r")
print("File as string:")
print(f1.read())
f1.close()

# 2) Read file line by line
f2 = open("C:\\R drive\\Python\\Files\\file1.txt", "r")
print("File line by line:")
for line in f2:
    print(line)
f2.close()

# 3) Read file as a list
f3 = open("C:\\R drive\\Python\\Files\\file1.txt", "r")
print("File as list:")
```

```
print(f3.readlines())
f3.close()
```

File as string:

This is file 1

File line by line:

This is 1st line from file1.txt

This is 2nd line

This is 3rd line

File as list:

```
['This is 1st line from file1.txt\n', 'This is 2nd line \n', 'This is 3rd line']
```

02) WAP to create file named "new.txt" only if it doesn't exist.

```
In [6]: f = open("new.txt", "w")
print("File new.txt created successfully")
f.close()
```

File new.txt created successfully

03) WAP to read first 5 lines from the text file.

```
In [13]: f = open("file1.txt", "r")
for i in range(5):
    print(f.readline())
f.close()
```

This is 1st line from file1.txt

This is 2nd line

This is 3rd line

This is 4th line

This is 5th line

04) WAP to find the longest word(s) in a file.

```
In [14]: f = open("file1.txt", "r")
words = f.read().split()
f.close()

longest = ""

for w in words:
    if len(w) > len(longest):
        longest = w

print("Longest word:", longest)
```

Longest word: thisIStheLONGESTwordINtheFILE

05) WAP to count the no. of lines, words and characters in a given text file.

```
In [15]: f = open("file1.txt", "r")

lines = f.readlines()
f.close()

line_count = len(lines)
word_count = 0
char_count = 0

for line in lines:
    word_count = word_count + len(line.split())
    char_count = char_count + len(line)

print("Number of lines:", line_count)
print("Number of words:", word_count)
print("Number of characters:", char_count)
```

Number of lines: 6
Number of words: 23
Number of characters: 130

06) WAP to copy the content of a file to the another file.

```
In [7]: f1 = open("file1.txt", "r")
data = f1.read()
f1.close()

f2 = open("copy.txt", "w")
f2.write(data)
f2.close()

print("File copied successfully")
```

File copied successfully

07) WAP to find the size of the text file.

```
In [9]: fp = open("file1.txt")
fp.read()
print("Total size = ", fp.tell(), "kb")
fp.close()
```

Total size = 140 kb

08) WAP to create an UDF named frequency to count occurrences of the specific word in a given text file.

```
In [19]: def frequency(word):
    f = open("file1.txt", "r")
    data = f.read()
    f.close()

    words = data.split()
```

```

count = 0

for w in words:
    if w == word:
        count = count + 1

return count

search = input("Enter word to search: ")
print("Frequency:", frequency(search))

```

Frequency: 5

09) WAP to get the score of five subjects from the user, store them in a file. Fetch those marks and find the highest score.

```

In [20]: f = open("marks.txt", "w")

for i in range(5):
    m = int(input("Enter marks: "))
    f.write(str(m) + "\n")

f.close()

f = open("marks.txt", "r")
marks = f.readlines()
f.close()

highest = int(marks[0])

for m in marks:
    if int(m) > highest:
        highest = int(m)

print("Highest score:", highest)

```

Highest score: 5

10) WAP to write first 100 prime numbers to a file named primenumbers.txt

(Note: each number should be in new line)

```

In [24]: f = open("primenumbers.txt", "w")
count = 0
num = 2
while count < 100:
    prime = True
    for i in range(2, num):
        if num % i == 0:
            prime = False
            break
    if prime:
        f.write(str(num) + "\n")
        count = count + 1

```

```
    num = num + 1
f.close()
```

11) WAP to merge two files and write it in a new file.

```
In [25]: f1 = open("file1.txt", "r")
data1 = f1.read()
f1.close()

f2 = open("file2.txt", "r")
data2 = f2.read()
f2.close()

f3 = open("merged.txt", "w")
f3.write(data1)
f3.write(data2)
f3.close()

print("Files merged successfully")
```

Files merged successfully

12) WAP to replace word1 by word2 of a text file. Write the updated data to new file.

```
In [26]: f = open("file1.txt", "r")
data = f.read()
f.close()

word1 = input("Enter word to replace: ")
word2 = input("Enter new word: ")

new_data = data.replace(word1, word2)

f2 = open("updated.txt", "w")
f2.write(new_data)
f2.close()

print("Word replaced and data written to new file")
```

Word replaced and data written to new file

13) Demonstrate tell() and seek() for all the cases(seek from beginning-end-current position) taking a suitable example of your choice.

seek

seek(offset, from_where) from_where: 0 - from beginning of the field 1 - from current position of the file pointer 2 - from end of the

```
In [19]: # f1 = open('file1.txt')
# f1.read(5)
# myLen = f1.tell()
```



```

# f1.seek(myLen,0)
# f1.seek(3,1)
# help(f1.seek)
# print(f1.tell())
# f1.close()

with open('seek_demo.txt', 'w') as f:
    f.write('helloworldfrompythonlab')

with open('seek_demo.txt', 'r') as f:
    print(f"Initial position: {f.tell()}")

    f.seek(0, 0)
    print(f"Position : {f.tell()}")
    f.read(5)
    f.seek(0, 1)
    print(f"Position: {f.tell()}")

    f.seek(0, 2)
    print(f"Position : {f.tell()}")

```

Initial position: 0
 Position : 0
 Position: 5
 Position : 23

```

In [11]: # Define Python user-defined exception
class NegativeNumberException(BaseException):
    "Raised when the input value is less than 0"
    pass

try:
    a = int(input("Enter Number: "))

    if a > 0:
        print(a)
    else:
        raise NegativeNumberException("Number is negative")

except NegativeNumberException as e:
    print(e)

```

Number is negative



Darshan
UNIVERSITY

Python Programming - 24010101107

Lab - 11

Roll No : 360

Name : Jadeja Vishwrajsinh

Progrmas based on Python Data Structure + UDF + File Handling

Practical 1: Count Lines, Words and Characters

Problem Definition:

Write a Python program to read text from a file and count the total number of lines, words, and characters. The result should be written to an output file.

```
In [5]: file1 = input("Enter input file name: ")
file2 = input("Enter output file name: ")

with open(file1, "r") as f:
    text = f.read()

lines = text.count("\n") + 1
words = len(text.split())
characters = len(text)

with open(file2, "w") as f:
    f.write(f"Lines: {lines}\n")
    f.write(f"Words: {words}\n")
    f.write(f"Characters: {characters}\n")

print("Count written to output file successfully!")
```

Count written to output file successfully!

Practical 2: Search a Word in a File

Problem Definition:

Write a Python program to search a given word in a text file and count its occurrences. Store the result in an output file.

```
In [7]: file1 = input("Enter input file name: ")
file3 = input("Enter output file name: ")
word = input("Enter word to search: ")

with open(file1, "r") as f:
    text = f.read()

count = text.split().count(word)

with open(file3, "w") as f:
    f.write(f"Word: {word}\n")
    f.write(f"Occurrences: {count}\n")

print("Result written to output file successfully!")
```

Result written to output file successfully!

Practical 3: Student Marks and Average

Problem Definition:

Write a Python program to read roll number, name, and marks of students from a file. Store marks in a list, calculate the average using a user-defined function, and write student details along with average marks to an output file.

```
In [17]: file4 = input("Enter input file name: ")
file5 = input("Enter output file name: ")

def average(marks):
    return sum(marks) / len(marks)

with open(file4, "r") as f:
    lines = f.readlines()

marks = []
students = []

for line in lines:
    roll, name, mark = line.split()
    mark = int(mark)
    marks.append(mark)
    students.append((roll, name, mark))

avg = average(marks)

with open(file5, "w") as f:
    for roll, name, mark in students:
        f.write(f"{roll} {name} {mark}\n")
    f.write(f"Average Marks: {avg}\n")
```

```
print("Student details and average written successfully!")
```

Student details and average written successfully!

Practical 4: Maximum and Minimum using Tuple

Problem Definition:

Write a Python program to read numbers from a file, store them in a tuple, and find the maximum and minimum values. Write the result to an output file.

```
In [18]: file6 = input("Enter input file name: ")
file7 = input("Enter output file name: ")

with open(file6, "r") as f:
    numbers = tuple(map(int, f.read().split()))

maximum = max(numbers)
minimum = min(numbers)

with open(file7, "w") as f:
    f.write(f"Numbers: {numbers}\n")
    f.write(f"Maximum: {maximum}\n")
    f.write(f"Minimum: {minimum}\n")

print("Maximum and minimum written successfully!")
```

Maximum and minimum written successfully!

Practical 5: Remove Duplicate Words using Set

Problem Definition:

Write a Python program to read words from a file and remove duplicate words using a set. Store the unique words in an output file.

```
In [19]: file1 = input("Enter input file name: ")
file8 = input("Enter output file name: ")

with open(file1, "r") as f:
    words = f.read().split()

unique_words = set(words)

with open(file8, "w") as f:
    for word in unique_words:
        f.write(word + "\n")

print("Duplicate words removed successfully!")
```

Duplicate words removed successfully!

Practical 6: Word Frequency using Dictionary

Problem Definition:

Write a Python program to count the frequency of each word in a text file using a dictionary and write the result to an output file.

```
In [20]: file1 = input("Enter input file name: ")
file9 = input("Enter output file name: ")

with open(file1, "r") as f:
    words = f.read().split()

freq = {}

for word in words:
    freq[word] = freq.get(word, 0) + 1

with open(file9, "w") as f:
    for word, count in freq.items():
        f.write(f"{word}: {count}\n")

print("Word frequency written successfully!")
```

Word frequency written successfully!

Practical 7: Student Result and Grade Calculation

Problem Definition:

Write a Python program to read roll number, name, and marks of students from a file.

Use a user-defined function to determine grade (A/B/C/Fail) and write the student result to an output file.

Grade Criteria:

- Marks ≥ 75 : Grade A
- Marks ≥ 60 : Grade B
- Marks ≥ 40 : Grade C
- Marks < 40 : Fail

```
In [22]: file4 = input("Enter input file name: ")
file10 = input("Enter output file name: ")

def grade(mark):
    if mark >= 75:
        return "A"
    elif mark >= 60:
        return "B"
    elif mark >= 40:
        return "C"
    else:
        return "Fail"

with open(file4, "r") as f:
    lines = f.readlines()
```

```
with open(file10, "w") as f:
    for line in lines:
        if not line.strip():
            continue
        roll, name, mark = line.split()
        mark = int(mark)
        g = grade(mark)
        f.write(f"{roll} {name} {mark} {g}\n")

print("Student results written successfully!")
```

Student results written successfully!

In []:



Darshan
UNIVERSITY

Python Programming - 2301CS404

Lab - 12

Roll No : 360

Name : Jadeja Vishwrajsinh

Exception Handling

01) WAP to handle following exceptions:

1. ZeroDivisionError
2. ValueError
3. TypeError

Note: handle them using separate except blocks and also using single except block too.

```
In [1]: try:
        num1 = int(input("Enter first number: "))
        num2 = int(input("Enter second number: "))

        result = num1 / num2
        print("Result:", result)

    except (ZeroDivisionError, ValueError, TypeError) as e:
        print("Error occurred:", e)
```

Error occurred: division by zero

02) WAP to handle following exceptions:

1. IndexError
2. KeyError

```
In [2]: try:
        numbers = [10, 20, 30, 40]
        index = int(input("Enter index number (0-3): "))
        print("Value at index:", numbers[index])

        student = {"name": "Rakshit", "age": 20}
        key = input("Enter key to search (name/age): ")
        print("Value for key:", student[key])

    except (IndexError, KeyError, ValueError) as e:
        print("Error occurred:", e)
```

Value at index: 20
Error occurred: '2'

03) WAP to handle following exceptions:

1. FileNotFoundError
2. ModuleNotFoundError

```
In [3]: try:
        filename = input("Enter file name to open: ")
        file = open(filename, "r")
        print(file.read())
        file.close()

        module_name = input("Enter module name to import: ")
        __import__(module_name)

        print("Module imported successfully!")

    except (FileNotFoundError, ModuleNotFoundError) as e:
        print("Error occurred:", e)
```

Error occurred: [Errno 2] No such file or directory: 'mytxt'

04) WAP that handles all type of exceptions in a single except block in standard error message format.

```
In [5]: try:
        num1 = int(input("Enter first number: "))
        num2 = int(input("Enter second number: "))

        result = num1 / num2
        print("Result:", result)

    except Exception as e:
        print("Error:", type(e).__name__, "-", e)
```

Error: ZeroDivisionError - division by zero

05) WAP to demonstrate else and finally block.

```
In [6]: # Program to demonstrate else and finally

        try:
            num1 = int(input("Enter first number: "))
```



```

num2 = int(input("Enter second number: "))

result = num1 / num2

except ZeroDivisionError:
    print("Error: Cannot divide by zero!")

except ValueError:
    print("Error: Please enter valid numbers!")

else:
    # Executes only if no exception occurs
    print("Result:", result)
    print("No errors occurred!")

finally:
    # Always executes
    print("Program execution completed.")

```

Error: Cannot divide by zero!
Program execution completed.

06) WAP to create an udf divide(a,b) that handles ZeroDivisionError.

In [7]: *# Function to divide two numbers*

```

def divide(a, b):
    try:
        result = a / b
        print("Result:", result)
    except ZeroDivisionError:
        print("Error: Cannot divide by zero!")

# Taking input from user
num1 = int(input("Enter first number: "))
num2 = int(input("Enter second number: "))

divide(num1, num2)

```

Error: Cannot divide by zero!

07) WAP to accept item prices and calculate total. Raise ValueError if price is negative, otherwise print the total bill amount.

In [8]: *# Program to calculate total bill amount*

```

total = 0

try:
    n = int(input("Enter number of items: "))

    for i in range(n):
        price = float(input(f"Enter price of item {i+1}: "))

        if price < 0:
            raise ValueError("Price cannot be negative!")

```

```

        total += price

    print("Total Bill Amount:", total)

except ValueError as e:
    print("Error:", e)

```

Total Bill Amount: 3.0

08) Create a short program that prompts the user for a list of grades separated by commas.

Split the string into individual grades and use a list comprehension to convert each string to an integer.

You should use a try statement to inform the user when the values they entered cannot be converted.

```

In [9]: # Program to convert grades into integers

try:
    grades_input = input("Enter grades separated by commas: ")

    grades = [int(g.strip()) for g in grades_input.split(",")]

    print("Grades List:", grades)

except ValueError:
    print("Error: Please enter only integer values separated by commas!")

```

Error: Please enter only integer values separated by commas!

09) Accept 5 subject marks (0-100). Use assert for validation.

Sample Input (Error Case)

Enter marks: 110

Sample Output (Error Case)

AssertionError: Marks should be between 0 and 100.

Sample Input (Normal Case)

Valid marks entered

Sample Output (Normal Case)

Average Marks displayed

```

In [10]: # Program to accept 5 subject marks and validate using assert

try:
    marks = []

    for i in range(5):
        m = int(input("Enter marks: "))
        assert 0 <= m <= 100, "Marks should be between 0 and 100."

```

```

marks.append(m)

print("Valid marks entered")
print("Average Marks:", sum(marks) / 5)

except AssertionError as e:
    print("AssertionError:", e)

except ValueError:
    print("Error: Please enter integer values only!")

```

Valid marks entered
Average Marks: 42.0

10) WAP that gets password from user. Password must have at least 8 characters and one digit.

Raise WeakPasswordError if invalid, print the message "Correct" otherwise.

```

In [11]: # Creating custom exception
class WeakPasswordError(Exception):
    pass

try:
    password = input("Enter password: ")

    # Check conditions
    if len(password) < 8 or not any(char.isdigit() for char in password):
        raise WeakPasswordError("Password must be at least 8 characters long and

    print("Correct")

except WeakPasswordError as e:
    print("WeakPasswordError:", e)

```

WeakPasswordError: Password must be at least 8 characters long and contain at least one digit.

11) WAP to raise your custom Exception named NegativeNumberError with the error message : "Cannot calculate the square root of a negative number" :

if the given number is negative.

otherwise print the square root of the given number.

```

In [13]: class NegativeNumberError(Exception):
        pass

import math

try:
    num = float(input("Enter a number: "))

    if num < 0:
        raise NegativeNumberError("Cannot calculate the square root of a negativ

```

```
print("Square Root:", math.sqrt(num))

except NegativeNumberError as e:
    print("NegativeNumberError:", e)

except ValueError:
    print("Error: Please enter a valid number!")
```

NegativeNumberError: Cannot calculate the square root of a negative number

In []:



Darshan
UNIVERSITY

Python Programming - 2301CS404

Lab - 13

Roll No. : 360

Name: Jadeja Vishwrajsinh

01) WAP to create Calculator module which defines functions like add, sub, mul and div.

Create another .py file that uses the functions available in Calculator module.

```
In [7]: import Calcy

print(Calcy.add(1,2))
print(Calcy.sub(1,2))
print(Calcy.mul(1,2))
print(Calcy.div(1,2))
```

0.5

02) WAP to pick a random character from a given String.

```
In [2]: import random
string = input("Enter a string: ")
text = random.choice(string)
print(text)
```

t

03) WAP to pick a random element from a given list.

```
In [6]: import random
list = ["Raksht", "is", "GOAT"]
something = random.choice(list)
print(something)
```

GOAT

04) WAP to roll a dice in such a way that every time you get the same number.

```
In [10]: import random
random.seed("sometext")
print(random.randint(1,2))
```

2

05) WAP to generate 3 random integers between 100 and 999 which is divisible by 5.

```
In [2]: from random import *
List = list(range(100,1000,5))
print(choice(List))
```

755

06) WAP to generate 100 random lottery tickets and pick two lucky tickets from it and announce them as Winner and Runner up respectively.

```
In [12]: import random
tickets = random.sample(range(1, 101), 100)
winner, runner_up = random.sample(tickets, 2)
print("\nWinner Ticket:", winner)
print("Runner-Up Ticket:", runner_up)
```

Winner Ticket: 49

Runner-Up Ticket: 65

07) WAP to print current date and time in Python.

```
In [4]: import datetime
print(datetime.datetime.now())
```

2026-03-31 04:48:04.574099

08) Subtract a week (7 days) from a given date in Python.

```
In [16]: from datetime import datetime, timedelta

date_input = input("Enter date (YYYY-MM-DD): ")
date_obj = datetime.strptime(date_input, "%Y-%m-%d")
new_date = date_obj - timedelta(days=7)

print("New Date:", new_date.date())
```

New Date: 2026-02-15

09) WAP to Calculate number of days between two given dates.

```
In [ ]: from datetime import datetime
```

```
date1 = input("Enter first date (YYYY-MM-DD): ")
date2 = input("Enter second date (YYYY-MM-DD): ")

d1 = datetime.strptime(date1, "%Y-%m-%d")
d2 = datetime.strptime(date2, "%Y-%m-%d")

print("Days Difference:", abs((d2 - d1).days))
```

10) WAP to Find the day of the week of a given date.(i.e. whether it is sunday/monday/tuesday/etc.)

```
In [ ]: from datetime import datetime

date_input = input("Enter date (YYYY-MM-DD): ")
date_obj = datetime.strptime(date_input, "%Y-%m-%d")

print("Day:", date_obj.strftime("%A"))
```



Darshan
UNIVERSITY

Python Programming - 2301CS404

Lab - 14

Roll No.: 360

Name: Jadeja Vishwrajsinh

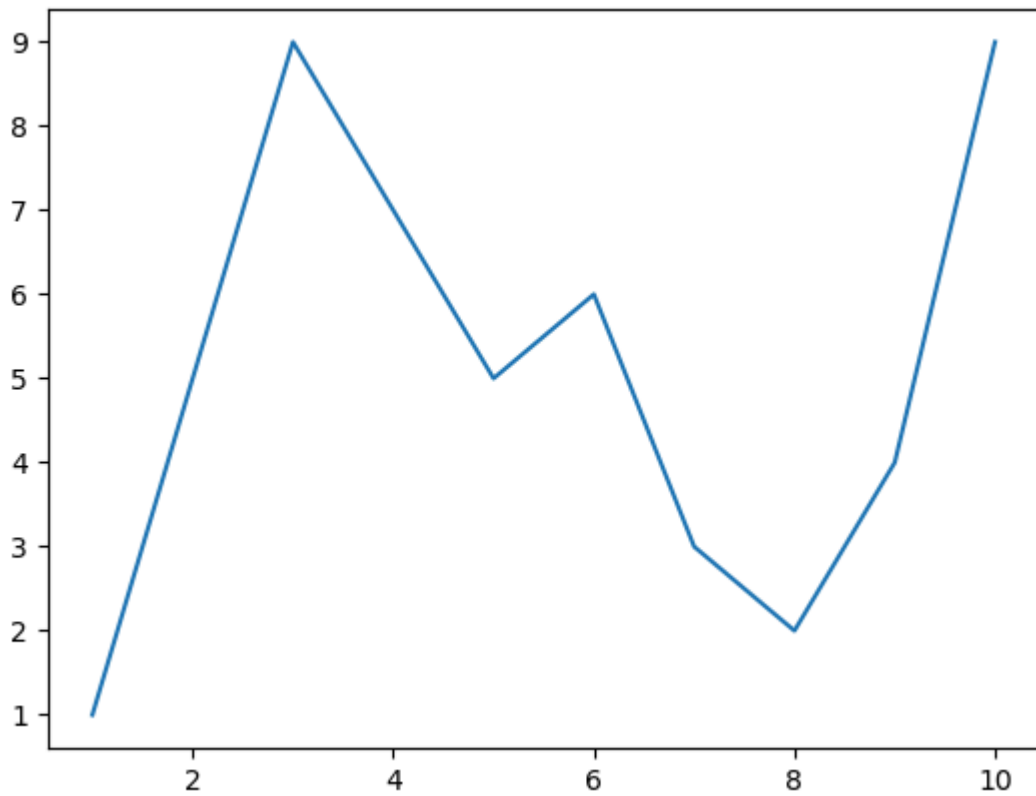
Import matplotlib below

```
In [1]: import matplotlib.pyplot as plt
```

01) WAP to display the line chart for given data for x and y axis.

```
In [7]: x = range(1,11)  
y = [1,5,9,7,5,6,3,2,4,9]
```

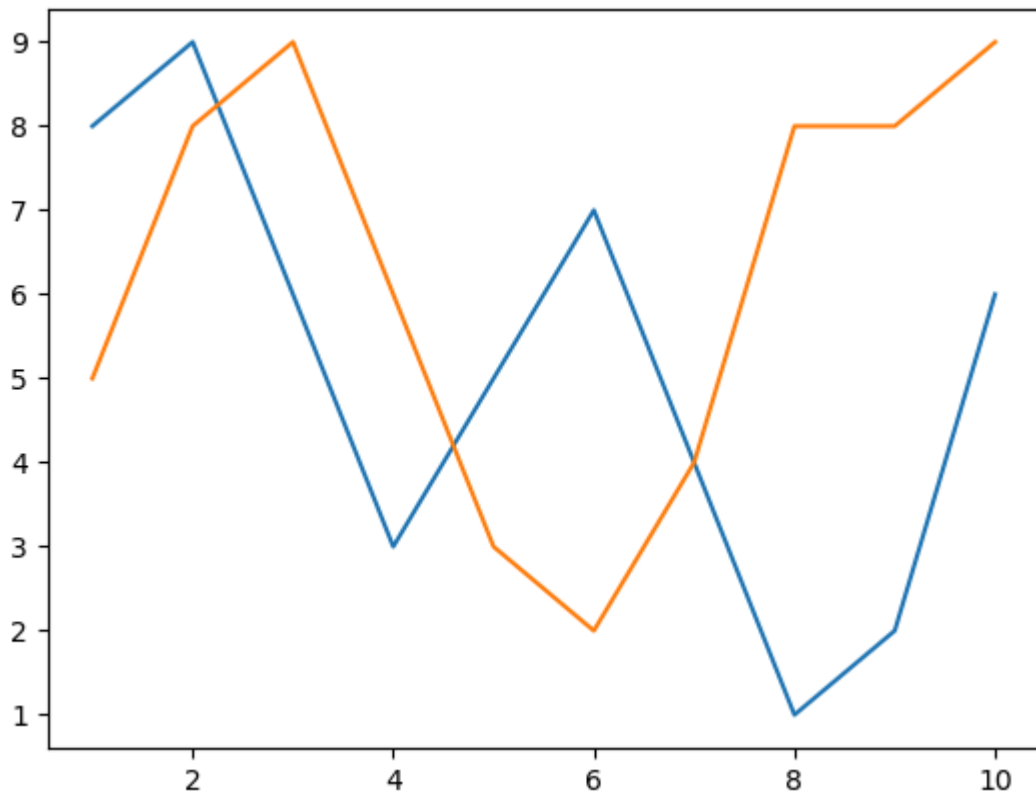
```
In [8]: plt.plot(x,y)  
plt.show()
```

02) WAP to generate line chart for given data.

```
In [9]: x = range(1,11,1)
cxMarks= [8,9,6,3,5,7,4,1,2,6]
cyMarks= [5,8,9,6,3,2,4,8,8,9]
```

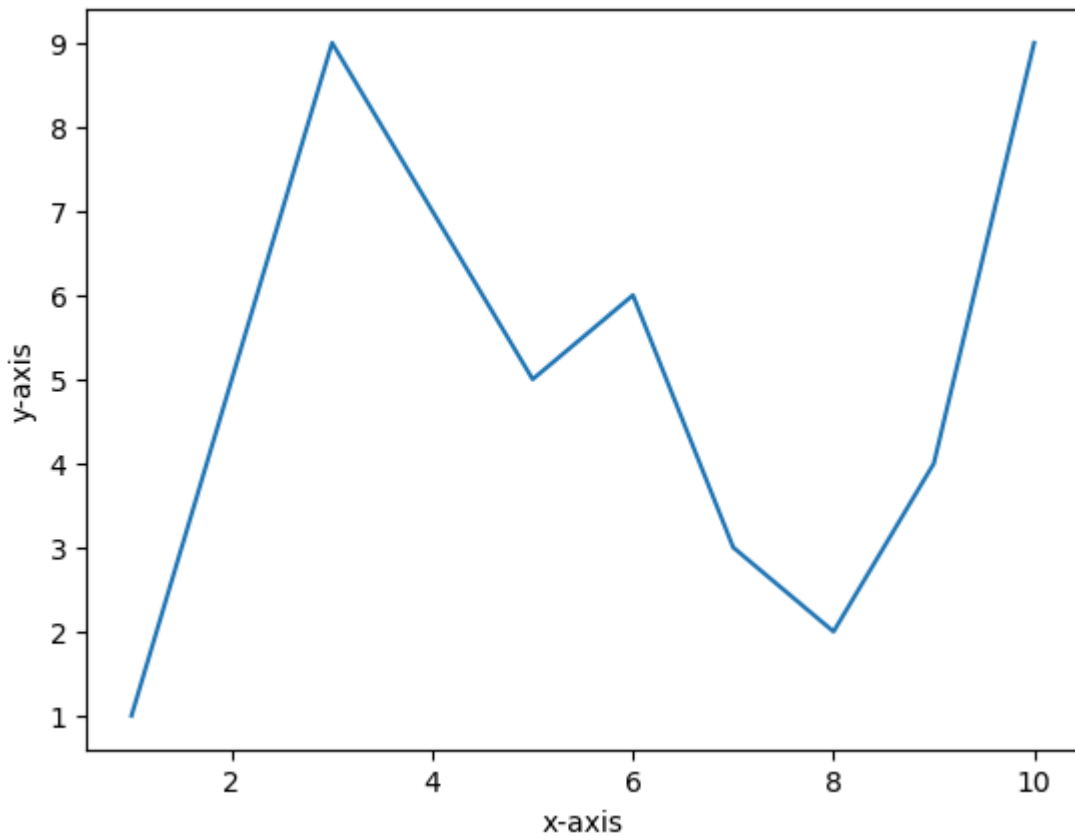
```
In [12]: plt.plot(x,cxMarks)
plt.plot(x,cyMarks)
plt.show()
```



03) WAP to generate line chart for given data, but create your custom axis.

```
In [ ]: x = range(1,11)
        y = [1,5,9,7,5,6,3,2,4,9]
```

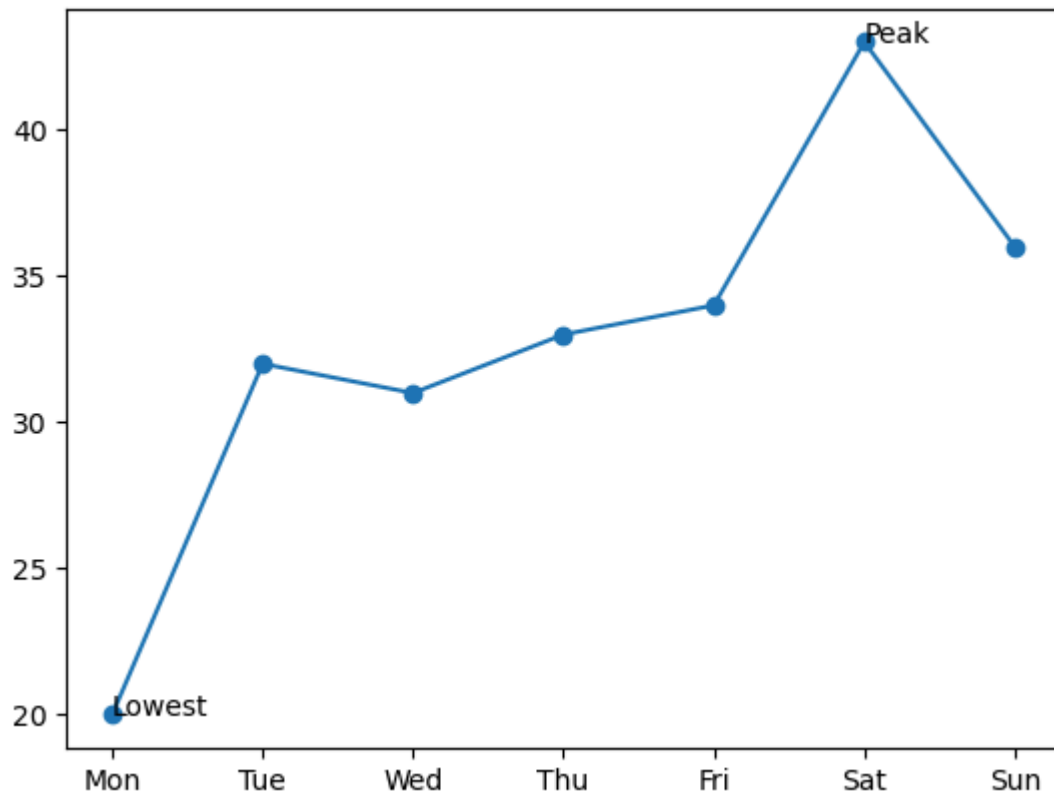
```
In [14]: plt.plot(x,y)
         plt.xlabel("x-axis")
         plt.ylabel("y-axis")
         plt.show()
```



04) WAP to plot a line chart showing temperature variation during a week. Mark the lowest and highest temperature.

```
In [5]: Days = ["Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun"]  
Temperature = [20, 32, 31, 33, 34, 43, 36]
```

```
In [9]: plt.plot(Days, Temperature, marker='o')  
plt.annotate('Lowest', xy=("Mon", 20))  
plt.annotate('Peak', xy=("Sat", 43))  
plt.show()
```

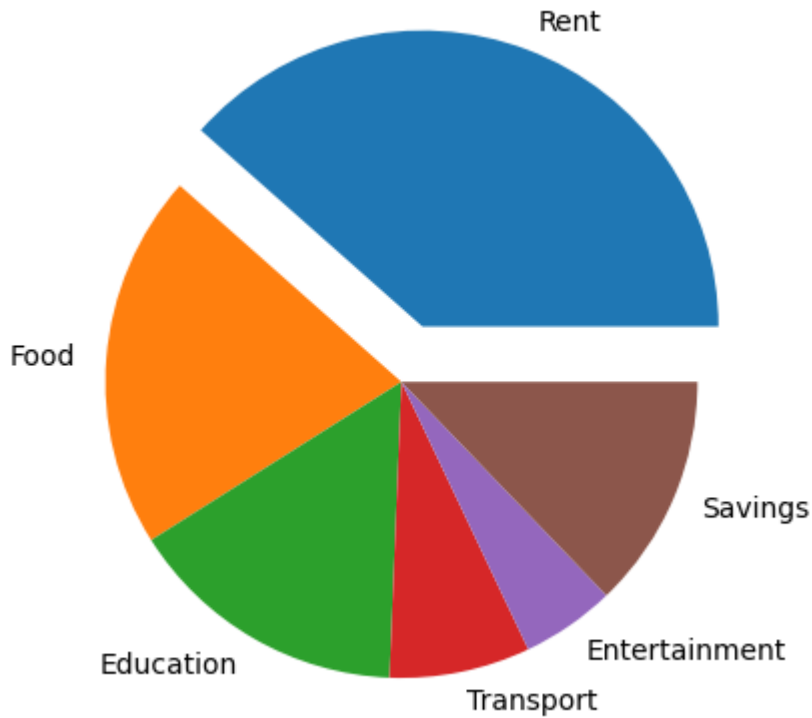


05) WAP to create a detailed pie chart representing the monthly expenditure distribution of a family.

The chart should display labels, percentage values, title, different colors for each slice, and explode one slice to highlight the highest expense.

```
In [12]: Categories = ["Rent", "Food", "Education", "Transport", "Entertainment", "Saving"]  
Amount = [15000, 8000, 6000, 3000, 2000, 5000]
```

```
In [16]: e = [0.2, 0, 0, 0, 0, 0]  
plt.pie(Amount, labels = Categories, explode=e)  
plt.show()
```



06) WAP to create bar chart showing the marks obtained by students in different subjects.

The program should also display the marks on the top of the respective bar.

```
In [17]: Subjects = ["Python", "DBMS", "SE", "PS", "PE-1"]
Marks = [78, 85, 72, 80, 90]
```

```
In [20]: import matplotlib.pyplot as plt

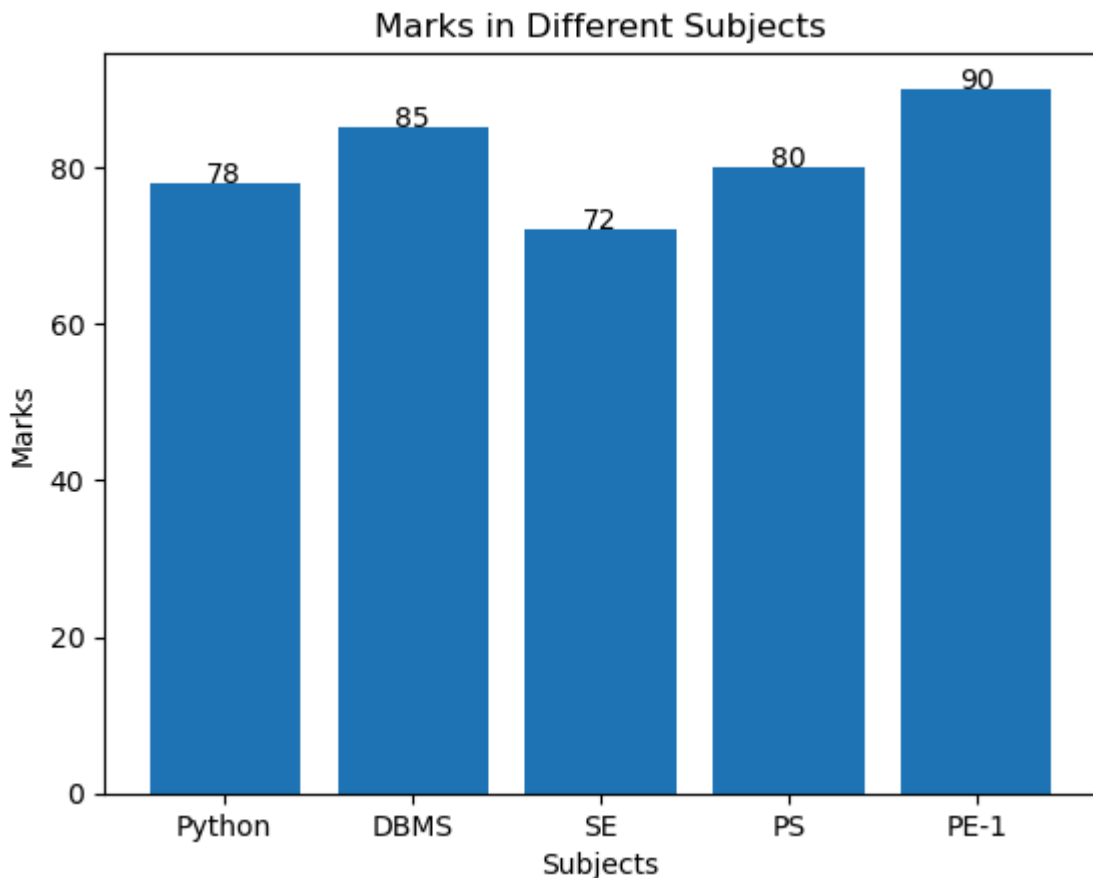
Subjects = ["Python", "DBMS", "SE", "PS", "PE-1"]
Marks = [78, 85, 72, 80, 90]

plt.bar(Subjects, Marks)

for i in range(len(Subjects)):
    plt.text(i, Marks[i], Marks[i], ha='center')

plt.xlabel("Subjects")
plt.ylabel("Marks")
plt.title("Marks in Different Subjects")

plt.show()
```



07) WAP to create a horizontal bar chart representing the number of books sold in different categories.

The program should also display the number of books sold at the end of the respective bar.

```
In [ ]: Categories = ["Fiction", "Science", "History", "Technology", "Biography"]
Books Sold = [120, 90, 70, 110, 60]
```

```
In [3]: import matplotlib.pyplot as plt

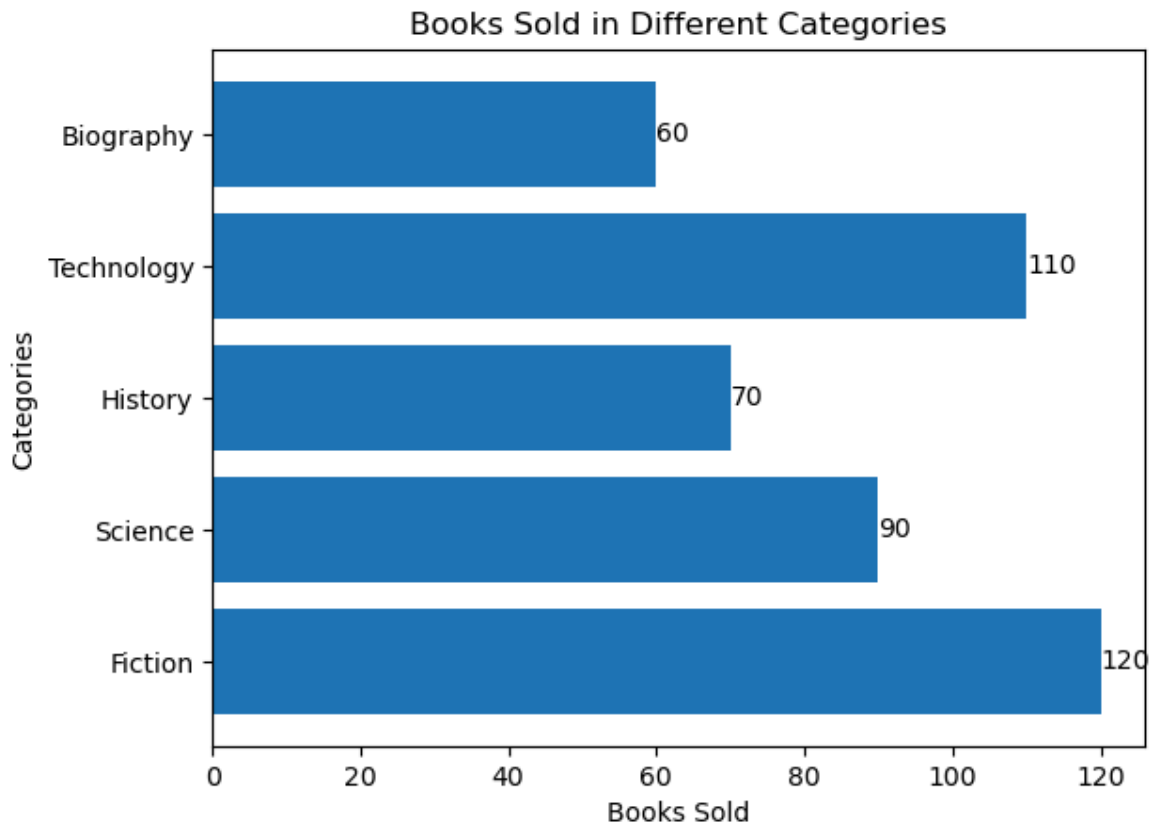
Categories = ["Fiction", "Science", "History", "Technology", "Biography"]
Books_Sold = [120, 90, 70, 110, 60]

plt.barh(Categories, Books_Sold)

for i in range(len(Categories)):
    plt.text(Books_Sold[i], i, Books_Sold[i], va='center')

plt.xlabel("Books Sold")
plt.ylabel("Categories")
plt.title("Books Sold in Different Categories")

plt.show()
```



08) WAP to create a scatter plot representing the relationship between advertising expenditure and sales revenue of a company.

```
In [ ]: Advertising Cost (₹ thousands) = [5, 8, 10, 12, 15, 18, 20]
Sales Revenue (₹ thousands) = [50, 60, 65, 70, 80, 90, 95]
```

```
In [4]: import matplotlib.pyplot as plt

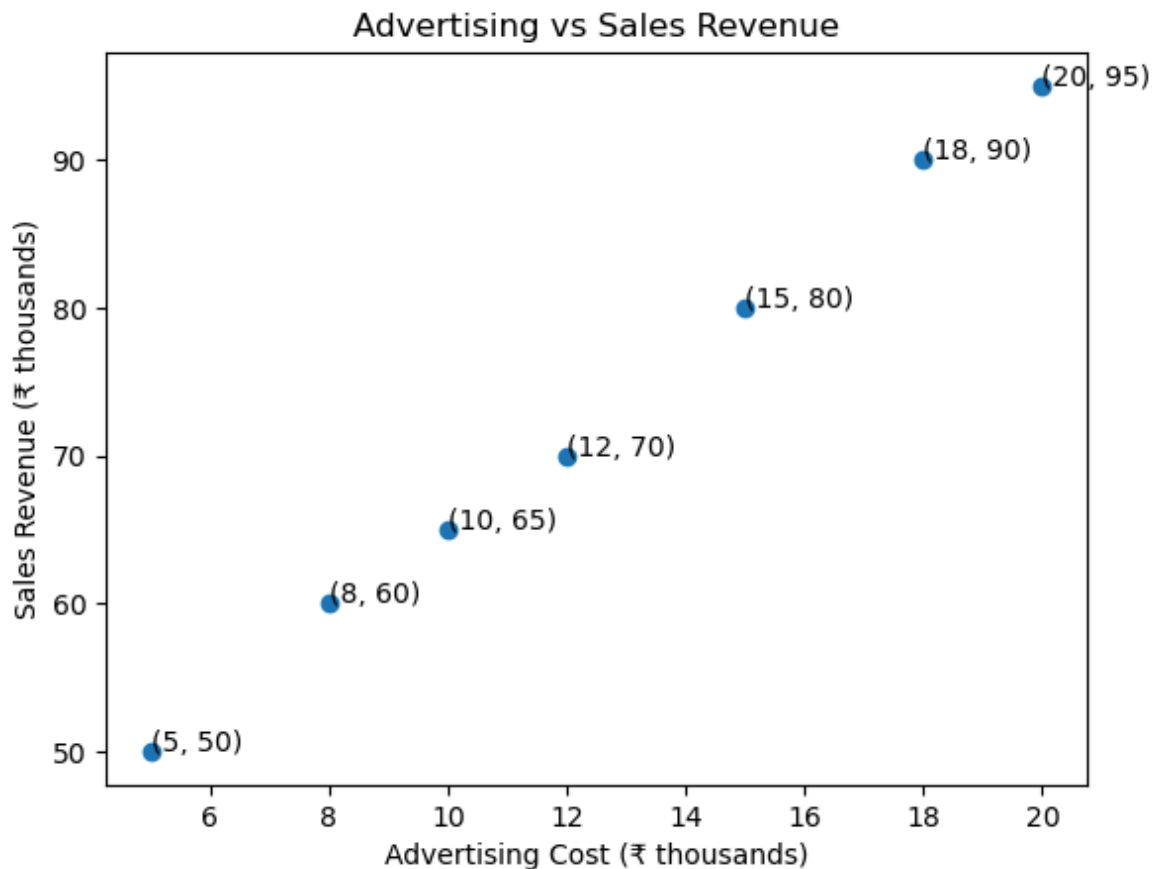
Advertising_Cost = [5, 8, 10, 12, 15, 18, 20]
Sales_Revenue = [50, 60, 65, 70, 80, 90, 95]

plt.scatter(Advertising_Cost, Sales_Revenue)

for i in range(len(Advertising_Cost)):
    plt.text(Advertising_Cost[i], Sales_Revenue[i], f"({Advertising_Cost[i]}, {S

plt.xlabel("Advertising Cost (₹ thousands)")
plt.ylabel("Sales Revenue (₹ thousands)")
plt.title("Advertising vs Sales Revenue")

plt.show()
```



09) WAP to create a histogram representing the distribution of marks obtained by students in a class.

```
In [ ]: Marks = [45, 50, 55, 60, 62, 65, 68, 70, 72, 75, 78, 80, 82, 85, 88, 90, 92, 95]
```

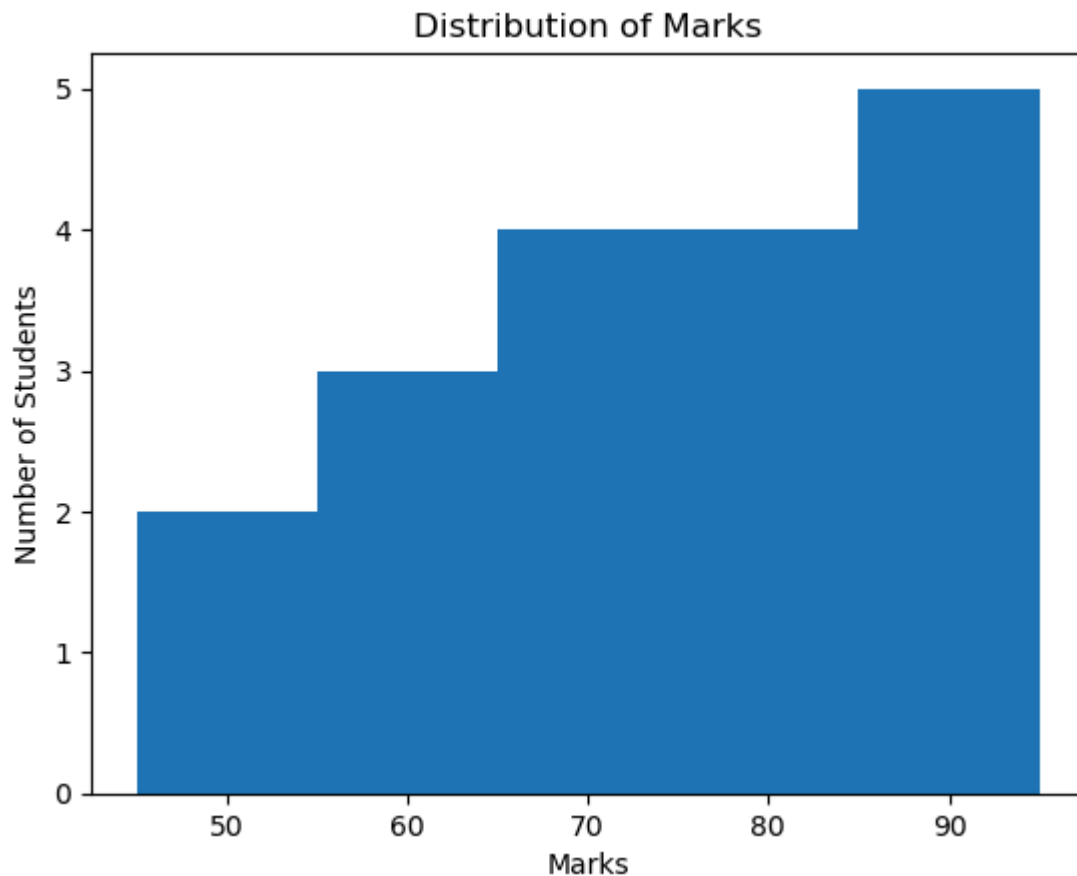
```
In [5]: import matplotlib.pyplot as plt

Marks = [45, 50, 55, 60, 62, 65, 68, 70, 72, 75, 78, 80, 82, 85, 88, 90, 92, 95]

plt.hist(Marks, bins=5)

plt.xlabel("Marks")
plt.ylabel("Number of Students")
plt.title("Distribution of Marks")

plt.show()
```

10) WAP to create a box plot representing the distribution of daily commuting time (in minutes) of employees and identify outliers (if any).

```
In [ ]: Commuting Time (minutes) = [5, 18, 20, 22, 25, 27, 28, 30, 32, 35, 38, 40, 42, 90]
```

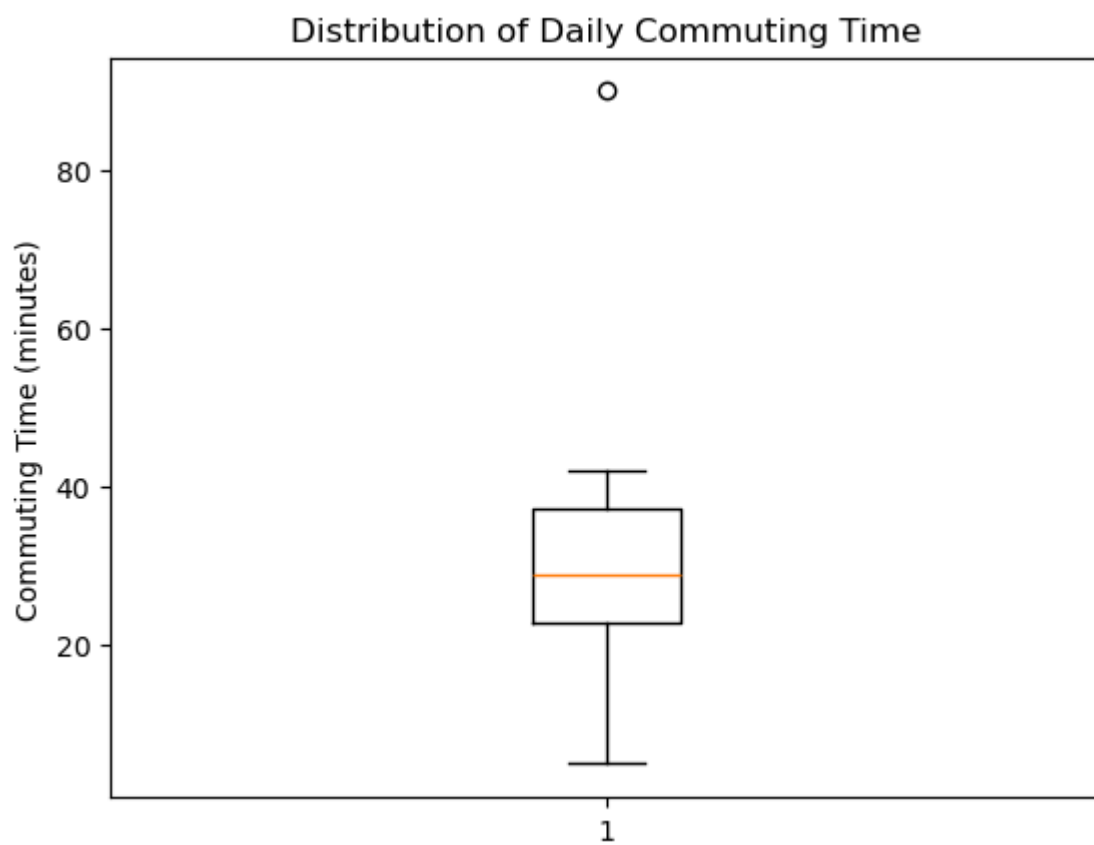
```
In [6]: import matplotlib.pyplot as plt

Commuting_Time = [5, 18, 20, 22, 25, 27, 28, 30, 32, 35, 38, 40, 42, 90]

plt.boxplot(Commuting_Time)

plt.ylabel("Commuting Time (minutes)")
plt.title("Distribution of Daily Commuting Time")

plt.show()
```



In []:



Darshan
UNIVERSITY

Python Programming - 2301CS404

Lab - 15 (1)

Roll No.: 360

Name: Jadeja Vishwrajsinh

01) Write a Program to create a class by name Students, and initialize attributes like name, age, and grade while creating an object.

```
In [1]: class Students:
        def __init__(self, name, age, grade):
            self.name = name
            self.age = age
            self.grade = grade

        name = input("Enter name: ")
        age = int(input("Enter age: "))
        grade = input("Enter grade: ")

        s1 = Students(name, age, grade)

        print("Name:", s1.name)
        print("Age:", s1.age)
        print("Grade:", s1.grade)
```

Name: Rakshit
Age: 20
Grade: 99

02) Create a class named Bank_Account with Account_No, User_Name, Email, Account_Type and Account_Balance data members. Also create a method GetAccountDetails() and DisplayAccountDetails(). Create main method to demonstrate the Bank_Account class.

```
In [3]: class Bank_Account:
        def __init__(self):
```

```

self.Account_No = ""
self.User_Name = ""
self.Email = ""
self.Account_Type = ""
self.Account_Balance = 0

def GetAccountDetails(self):
    self.Account_No = input("Enter Account No: ")
    self.User_Name = input("Enter User Name: ")
    self.Email = input("Enter Email: ")
    self.Account_Type = input("Enter Account Type: ")
    self.Account_Balance = float(input("Enter Account Balance: "))

def DisplayAccountDetails(self):
    print("Account No:", self.Account_No)
    print("User Name:", self.User_Name)
    print("Email:", self.Email)
    print("Account Type:", self.Account_Type)
    print("Account Balance:", self.Account_Balance)

def main():
    acc = Bank_Account()
    acc.GetAccountDetails()
    acc.DisplayAccountDetails()

main()

```

Account No: 123456789
 User Name: Rakshit
 Email: rakshit@darshan.ac.in
 Account Type: Main
 Account Balance: 999999999.0

03) WAP to create Circle class with area and perimeter function to find area and perimeter of circle.

```

In [2]: class Circle:
        def __init__(self, radius):
            self.radius = radius

        def area(self):
            return 3.14 * self.radius * self.radius

        def perimeter(self):
            return 2 * 3.14 * self.radius

r = float(input("Enter radius: "))
c = Circle(r)

print("Area:", c.area())
print("Perimeter:", c.perimeter())

```

Area: 3.14
 Perimeter: 6.28

04) Create a class for employees that includes attributes such as name, age, salary, and methods to update and display employee information.

```
In [4]: class Employee:
    def __init__(self, name, age, salary):
        self.name = name
        self.age = age
        self.salary = salary

    def update(self):
        self.name = input("Enter new name: ")
        self.age = int(input("Enter new age: "))
        self.salary = float(input("Enter new salary: "))

    def display(self):
        print("Name:", self.name)
        print("Age:", self.age)
        print("Salary:", self.salary)

name = input("Enter name: ")
age = int(input("Enter age: "))
salary = float(input("Enter salary: "))

emp = Employee(name, age, salary)
emp.display()

emp.update()
emp.display()
```

Name: Rakshit
 Age: 20
 Salary: 99999.0
 Name: Ruchit
 Age: 19
 Salary: 99.0

05) Create a bank account class with methods to deposit, withdraw, and check balance.

```
In [5]: class BankAccount:
    def __init__(self, balance):
        self.balance = balance

    def deposit(self, amount):
        self.balance += amount

    def withdraw(self, amount):
        if amount <= self.balance:
            self.balance -= amount
        else:
            print("Insufficient Balance")

    def check_balance(self):
        print("Balance:", self.balance)

balance = float(input("Enter initial balance: "))
acc = BankAccount(balance)

amount = float(input("Enter deposit amount: "))
acc.deposit(amount)
acc.check_balance()
```

```
amount = float(input("Enter withdraw amount: "))
acc.withdraw(amount)
acc.check_balance()
```

Balance: 20000.0

Balance: 10000.0

06) Create a class for managing inventory that includes attributes such as item name, price, quantity, and methods to add, remove, and update items.

```
In [6]: class Inventory:
        def __init__(self, name, price, quantity):
            self.name = name
            self.price = price
            self.quantity = quantity

        def add(self, qty):
            self.quantity += qty

        def remove(self, qty):
            if qty <= self.quantity:
                self.quantity -= qty
            else:
                print("Not enough stock")

        def update(self, price):
            self.price = price

        def display(self):
            print("Item:", self.name)
            print("Price:", self.price)
            print("Quantity:", self.quantity)

name = input("Enter item name: ")
price = float(input("Enter price: "))
quantity = int(input("Enter quantity: "))

item = Inventory(name, price, quantity)
item.display()

qty = int(input("Enter quantity to add: "))
item.add(qty)
item.display()

qty = int(input("Enter quantity to remove: "))
item.remove(qty)
item.display()

new_price = float(input("Enter new price: "))
item.update(new_price)
item.display()
```

Item: Book

Price: 20.0

Quantity: 2

Item: Book

Price: 20.0

Quantity: 4

Item: Book
Price: 20.0
Quantity: 2
Item: Book
Price: 20.0
Quantity: 2

07) Create a Class with instance attributes of your choice.

```
In [7]: class Car:
        def __init__(self, brand, model, year):
            self.brand = brand
            self.model = model
            self.year = year

        def display(self):
            print("Brand:", self.brand)
            print("Model:", self.model)
            print("Year:", self.year)

        brand = input("Enter brand: ")
        model = input("Enter model: ")
        year = int(input("Enter year: "))

        c = Car(brand, model, year)
        c.display()
```

Brand: BMW
Model: M5
Year: 2020

08) Create one class student_kit

Within the student_kit class create one class attribute principal name (Mr ABC)

Create one attendance method and take input as number of days.

While creating student take input their name .

Create one certificate for each student by taking input of number of days present in class.

```
In [8]: class student_kit:
        principal_name = "Mr ABC"

        def __init__(self, name):
            self.name = name

        def attendance(self, days):
            self.days = days

        def certificate(self):
            print("Principal:", student_kit.principal_name)
            print("Student Name:", self.name)
            print("Days Present:", self.days)

        name = input("Enter student name: ")
```

```
s = student_kit(name)

days = int(input("Enter number of days present: "))
s.attendance(days)

s.certificate()
```

Principal: Mr ABC

Student Name: Rakshit

Days Present: 22

09) Define Time class with hour and minute as data member. Also define addition method to add two time objects.

```
In [7]: class Time:
        def __init__(self, hour, minute):
            self.hour = hour
            self.minute = minute

        def add(self, t):
            total_m = self.minute + t.minute
            extra_hr = total_m // 60
            final_min = total_m % 60
            final_hr = self.hour + t.hour + extra_hr
            return Time(final_hr, final_min)

t1 = Time(2, 50)
t2 = Time(1, 30)

t3 = t1.add(t2)
print(t3.hour, "hour", t3.minute, "minute")
```

4 hour(s) 20 minute(s)

In []:

In []:



Darshan
UNIVERSITY

Python Programming - 2301CS404

Lab - 15 (2)

Roll No. : 360

Name : Jadeja Vishwrajsinh

Continued..

10) Calculate area of a rectangle using object as an argument to a method.

```
In [13]: class Rectangle:
          def __init__(self, l, b):
              self.length = l
              self.breath = b

          def area(self):
              return self.length * self.breath

r1 = Rectangle(5,10)

result = Rectangle.area(r1)
print(f"Area of rectangle: {result}")
```

Area of rectangle: 50

11) Calculate the area of a square.

Include a Constructor, a method to calculate area named `area()` and a method named `output()` that prints the output and is invoked by `area()`.

```
In [11]: class Square:
          def __init__(self, l):
              self.length = l

          def area(self):
```

```

        area = self.length * self.length
        self.output(area)

    def output(self, area):
        print(f"Area of square: {area}")

s1 = Square(5)
s1.area()

```

Area of square: 25

12) Calculate the area of a rectangle.

Include a Constructor, a method to calculate area named `area()` and a method named `output()` that prints the output and is invoked by `area()`.

Also define a class method that compares the two sides of reactangle. An object is instantiated only if the two sides are different; otherwise a message should be displayed : THIS IS SQUARE.

```

In [31]: class Rectangle:
    def __init__(self, l, b):
        self.length = l
        self.breath = b

    @classmethod
    def area(cls, obj):
        if obj.length != obj.breath:
            result = obj.length * obj.breath
            obj.output(result)
        else:
            print("THIS IS SQUARE")

    def output(self, result):
        print(f"Area of rectangle: {result}")

r2 = Rectangle(5,5)
Rectangle.area(r2)

```

THIS IS SQUARE

13) Define a class Square having a private attribute "side".

Implement `get_side` and `set_side` methods to acceses the private attribute from outside of the class.

```

In [35]: class Square:
    def __init__(self, s1):
        self.__side = s1

    def get_side(self):
        print(f"Square side is: {self.__side}")

```

```

def set_side(self, s2):
    self.__side = s2
    print(f"Square's updated side is: {self.__side}")

s2 = Square(10)
s2.get_side()
s2.set_side(15)

```

Square side is: 10

Square's updated side is: 15

14) Create a class Profit that has a method named getProfit that accepts profit from the user.

Create a class Loss that has a method named getLoss that accepts loss from the user.

Create a class BalanceSheet that inherits from both classes Profit and Loss and calculates the balance. It has two methods getBalance() and printBalance().

```

In [37]: class Profit:
def __init__(self):
    self.profit = 0

def get_profit(self):
    self.profit = int(input("Enter profit amount: "))

class loss:
def __init__(self):
    self.loss = 0

def get_loss(self):
    self.loss = int(input("Enter loss amount: "))

class BalanceSheet(Profit, loss):
def getBalance(self):
    self.balance = self.profit - self.loss

def printBalance(self):
    print(f"Balance is: {self.balance}")

b1 = BalanceSheet()
b1.get_profit()
b1.get_loss()
b1.getBalance()
b1.printBalance()

```

Balance is: 500

15) WAP to demonstrate all types of inheritance.

```

In [1]: # 1. Single Inheritance
class A:
def show_A(self):
    print("Class A")

```

```
class B(A):
    def show_B(self):
        print("Class B (Single Inheritance)")

# 2. Multiple Inheritance
class C:
    def show_C(self):
        print("Class C")

class D:
    def show_D(self):
        print("Class D")

class E(C, D):
    def show_E(self):
        print("Class E (Multiple Inheritance)")

# 3. Multilevel Inheritance
class F:
    def show_F(self):
        print("Class F")

class G(F):
    def show_G(self):
        print("Class G")

class H(G):
    def show_H(self):
        print("Class H (Multilevel Inheritance)")

# 4. Hierarchical Inheritance
class I:
    def show_I(self):
        print("Class I")

class J(I):
    def show_J(self):
        print("Class J")

class K(I):
    def show_K(self):
        print("Class K (Hierarchical Inheritance)")

# 5. Hybrid Inheritance (Combination)
class L:
    def show_L(self):
        print("Class L")

class M(L):
    def show_M(self):
        print("Class M")

class N(L):
    def show_N(self):
        print("Class N")
```

```
class O(M, N):
    def show_O(self):
        print("Class O (Hybrid Inheritance)")

print("\nSingle Inheritance:")
b = B()
b.show_A()
b.show_B()

print("\nMultiple Inheritance:")
e = E()
e.show_C()
e.show_D()
e.show_E()

print("\nMultilevel Inheritance:")
h = H()
h.show_F()
h.show_G()
h.show_H()

print("\nHierarchical Inheritance:")
j = J()
k = K()
j.show_I()
j.show_J()
k.show_I()
k.show_K()

print("\nHybrid Inheritance:")
o = O()
o.show_L()
o.show_M()
o.show_N()
o.show_O()
```

Single Inheritance:
Class A
Class B (Single Inheritance)

Multiple Inheritance:
Class C
Class D
Class E (Multiple Inheritance)

Multilevel Inheritance:
Class F
Class G
Class H (Multilevel Inheritance)

Hierarchical Inheritance:
Class I
Class J
Class I
Class K (Hierarchical Inheritance)

Hybrid Inheritance:
Class L
Class M
Class N
Class O (Hybrid Inheritance)

16) Create a Person class with a constructor that takes two arguments name and age.

Create a child class Employee that inherits from Person and adds a new attribute salary.

Override the `init` method in Employee to call the parent class's `init` method using the `super()` and then initialize the salary attribute.

```
In [1]: class Person:
        def __init__(self, name, age):
            self.name = name
            self.age = age

        def show(self):
            print(f"Name: {self.name}")
            print(f"Age: {self.age}")

        class Employee(Person):
            def __init__(self, name, age, salary):
                super().__init__(name, age)
                self.salary = salary

            def show(self):
                print(f"Salary: {self.salary}")

        e1 = Employee("Rakshit", 20, 100000)
        e1.show()
```

Salary: 100000

17) Create a Shape class with a draw method that is not implemented.

Create three child classes Rectangle, Circle, and Triangle that implement the draw method with their respective drawing behaviors.

Create a list of Shape objects that includes one instance of each child class, and then iterate through the list and call the draw method on each object.

```
In [2]: # Base class
class Shape:
    def draw(self):
        raise NotImplementedError("Subclass must implement draw method")

# Child class: Rectangle
class Rectangle(Shape):
    def draw(self):
        print("Drawing Rectangle")

# Child class: Circle
class Circle(Shape):
    def draw(self):
        print("Drawing Circle")

# Child class: Triangle
class Triangle(Shape):
    def draw(self):
        print("Drawing Triangle")

# Create list of Shape objects
shapes = [Rectangle(), Circle(), Triangle()]

# Iterate and call draw()
for shape in shapes:
    shape.draw()
```

```
Drawing Rectangle
Drawing Circle
Drawing Triangle
```